

LEZIONI 37-38

Architetture MVC/MVP

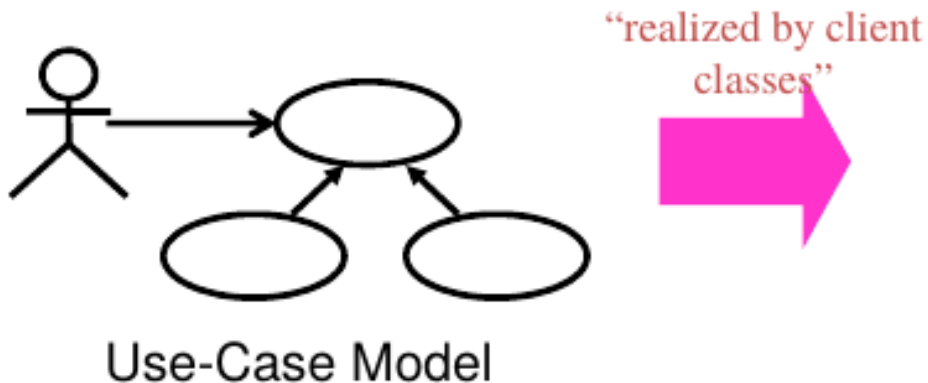
Schemi ed implicazioni per architetture centralizzate e distribuite

Ingegneria del Software e Progettazione Web
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

design guidato dalle responsabilità

Required Functionality



Analysis Classes

Entity Classes

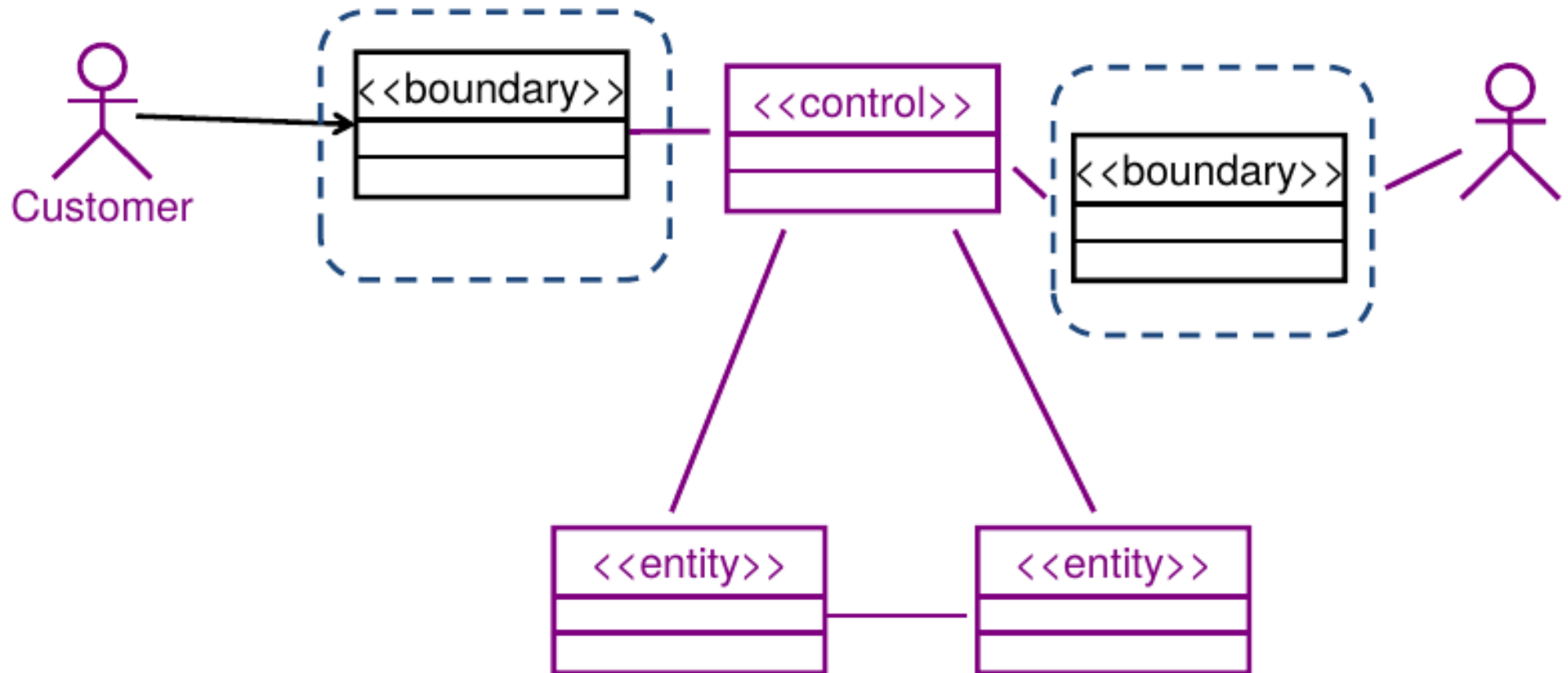
Control Classes

Boundary Classes

BCE : quali responsabilità?

- **boundary**: mediano l'interazione tra il sistema e l'ambiente
 - rappresentano elementi al confine del sistema
 - regolano l'interazione con gli attori
- **control**: coordinano il comportamento durante un caso d'uso del sistema
 - rappresentano comportamenti dipendenti dall'interazione attesa con il sistema (i.e. comportamento descritto dal caso d'uso)
 - sono indipendenti dal modo di attivazione dell'interazione
 - queste classi dovrebbero scaturire dall'analisi dominio del problema
- **entity**: rappresentano le astrazioni chiave del sistema, cioè trasposizioni logiche degli elementi reali del dominio considerato
 - sono indipendenti dall'ambiente di esecuzione
 - dipendono dall'analisi dominio del problema
 - glossario, use case, business model, etc.
 - modellano il comportamento di una entità di dominio incapsulando un insieme coeso di dati
 - (solo in prima approssimazione,) definiscono i tipi di dati che vengono manipolati dall'insieme dei controller ed i cui valori sono esposti/richiesti dalle boundary

schema di interazioni in BCE



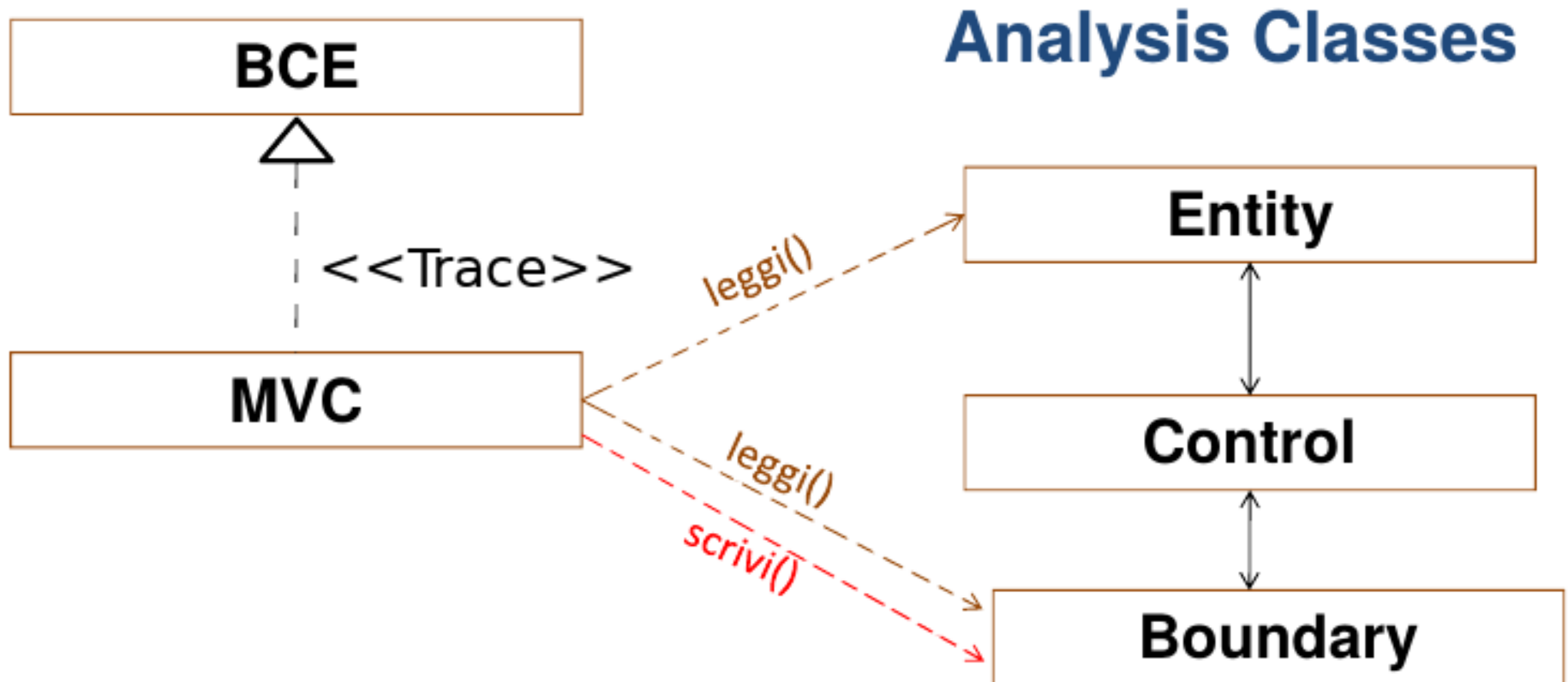
... oltre BCE

- BCE è un pattern di analisi per sistemi O.O.
 - lo scopo principale è identificare le macro classi del sistema
 - le classi derivano dall'osservazione:
 - del dominio
 - dai modi in cui il sistema è utilizzato dagli attori (i.e. casi d'uso)
- come raffinare un modello di analisi BCE?
- come gestire/introdurre aspetti:
 - nel dominio della soluzione
 - che tengano conto di aspetti di ingegnerizzazione del sistema (e.g. riuso, ottimizzazione delle soluzioni, manutenibilità, scalabilità, etc.)

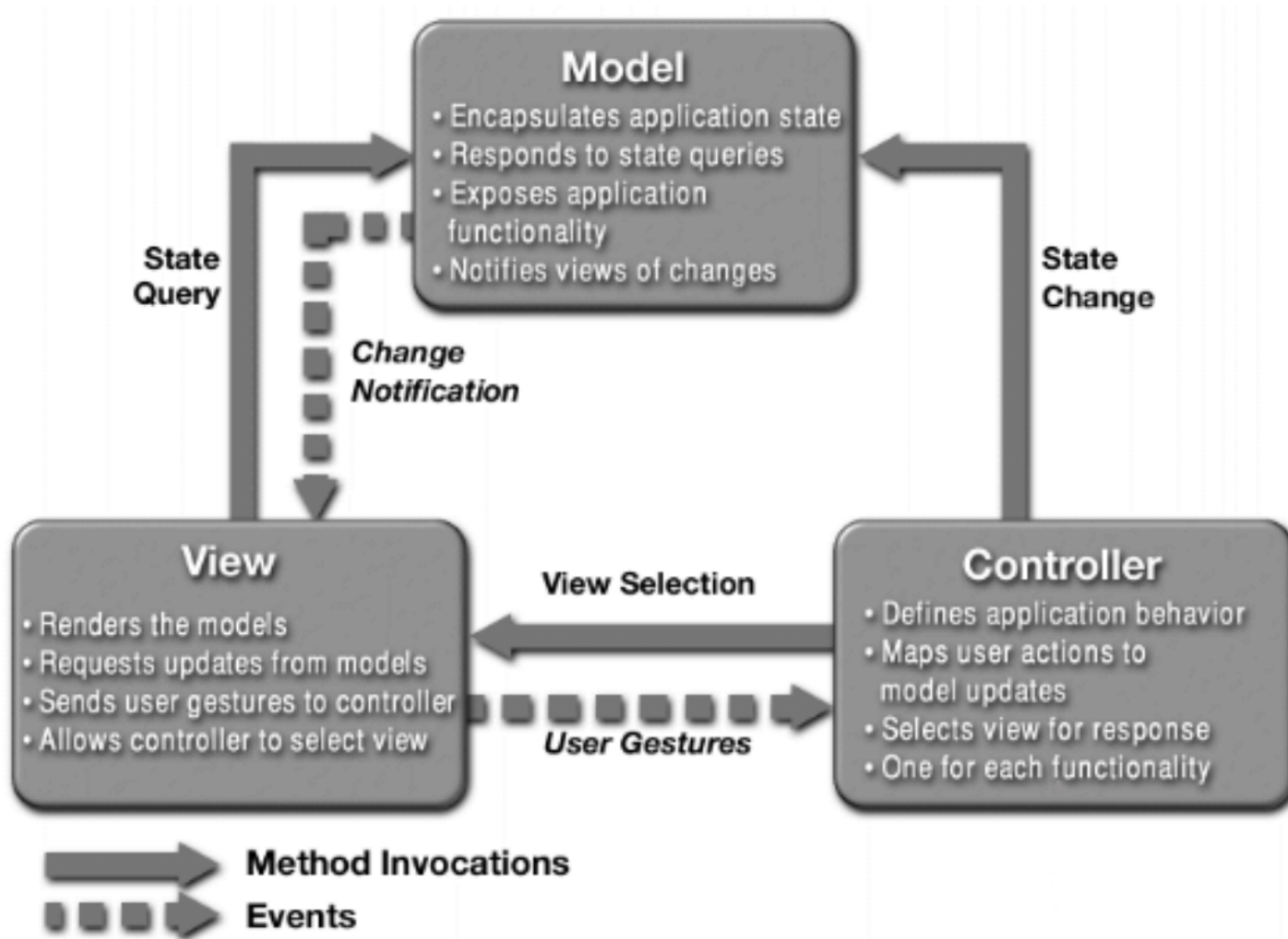
... oltre BCE

- BCE è un pattern di analisi per sistemi O.O.
 - lo scopo principale è identificare le macro classi del sistema
 - le classi derivano dall'osservazione:
 - del dominio
 - dai modi in cui il sistema è utilizzato dagli attori (i.e. casi d'uso)
- come raffinare un modello di analisi BCE?
- come gestire/introdurre aspetti:
 - nel dominio della soluzione
 - che tengano conto di aspetti di ingegnerizzazione del sistema (e.g. riuso, ottimizzazione delle soluzioni, manutenibilità, scalabilità, etc.)
- il pattern di analisi BCE può essere raffinato secondo diverse varianti:
 - Model-View-Controller (MVC)
 - Model-View-Presenter (MVP)

model-view-controller



model-view-controller



MVC – view

- responsabile per la
 - la logica di presentazione dei dati
 - la costruzione/gestione dell'interfaccia grafica (GUI)
 - acquisizione dei dati dell'applicazione
 - (se non ne è responsabile il del controller) può gestire la conversione dei dati da formato esterno a interno (e viceversa)
 - “12 Ottobre 1942” → {1492; 10; 12}
 - accedere ai dati in SOLA lettura direttamente al Model (attributi)
- mantenere aggiornati i dati presentati
 - push model: applicazione del GoF Observer
 - le View possono registrarsi come osservatori di Model
 - le View ricevono aggiornamenti di Model in “tempo reale”
 - pull model: utilizzato nel caso in cui la View intende richiedere gli aggiornamenti solo quando “opportuno”

MVC – controller

- controller grafico dell'applicazione: coordina l'interazione tra View ed il Controller Applicativo (o il Model in sola lettura) per la realizzazione di una funzionalità
 - converte formati esterno in interno e viceversa (se non a carico della View)
 - realizza la mappa tra input dell'utente e processi eseguiti dal Model
 - crea/seleziona le istanze di View richieste
- controller applicativo: implementa la logica di controllo dell'applicazione
 - raffina il concetto di **Controller in BCE**
 - ha la responsabilità di gestire una azione dell'utente (sulla View) in una o più azioni eseguite sulle istanze nel Model
 - per quanto possibile data l'applicazione dovrebbe avere un comportamento state-less
 - interazioni differenti sul controller, a parità di (tutti gli) input, devono tornare lo stesso risultato
 - il controller applicativo (di per se) non ha memoria:
 - non dovrebbe esistere uno stato comune tra le istanze
 - attivazioni diverse dello stesso caso d'uso fanno riferimento a diverse istanze di controller applicativo
- progettazione ed implementazione:
 - controller grafico ed applicativo aggregati in un unico “bundle”
 - il controller grafico implementa direttamente la logica di controllo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono anche sul controller
 - controller grafico ed applicativo disaccoppiati
 - il controller grafico viene associato a parte della view
 - il controller grafico invoca operazioni sul controller applicativo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono solo sul controller grafico e non sul modo in cui il caso d'uso è realizzato (i.e. controller applicativo)

MVC – controller

- controller grafico dell'applicazione: coordina l'interazione tra View ed il Controller Applicativo (o il Model in sola lettura) per la realizzazione di una funzionalità
 - converte formati esterno in interno e viceversa (se non a carico della View)
 - realizza la mappa tra input dell'utente e processi eseguiti dal Model
 - crea/seleziona le istanze di View richieste
- controller applicativo: implementa la logica di controllo dell'applicazione
 - raffina il concetto di **Controller in BCE**
 - - quale soluzione prediligiamo in questo corso?
 - perché?
 - in preparazione dell'esame, argomentare le risposte
 - non dovrebbe esistere uno stato comune tra le istanze
 - attivazioni diverse dello stesso caso d'uso fanno riferimento a diverse istanze di controller applicativo
- progettazione ed implementazione:
 - controller grafico ed applicativo aggregati in un unico “bundle”
 - il controller grafico implementa direttamente la logica di controllo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono anche sul controller
 - controller grafico ed applicativo disaccoppiati
 - il controller grafico viene associato a parte della view
 - il controller grafico invoca operazioni sul controller applicativo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono solo sul controller applicativo e non sul modo in cui il caso d'uso è realizzato (i.e. controller applicativo)

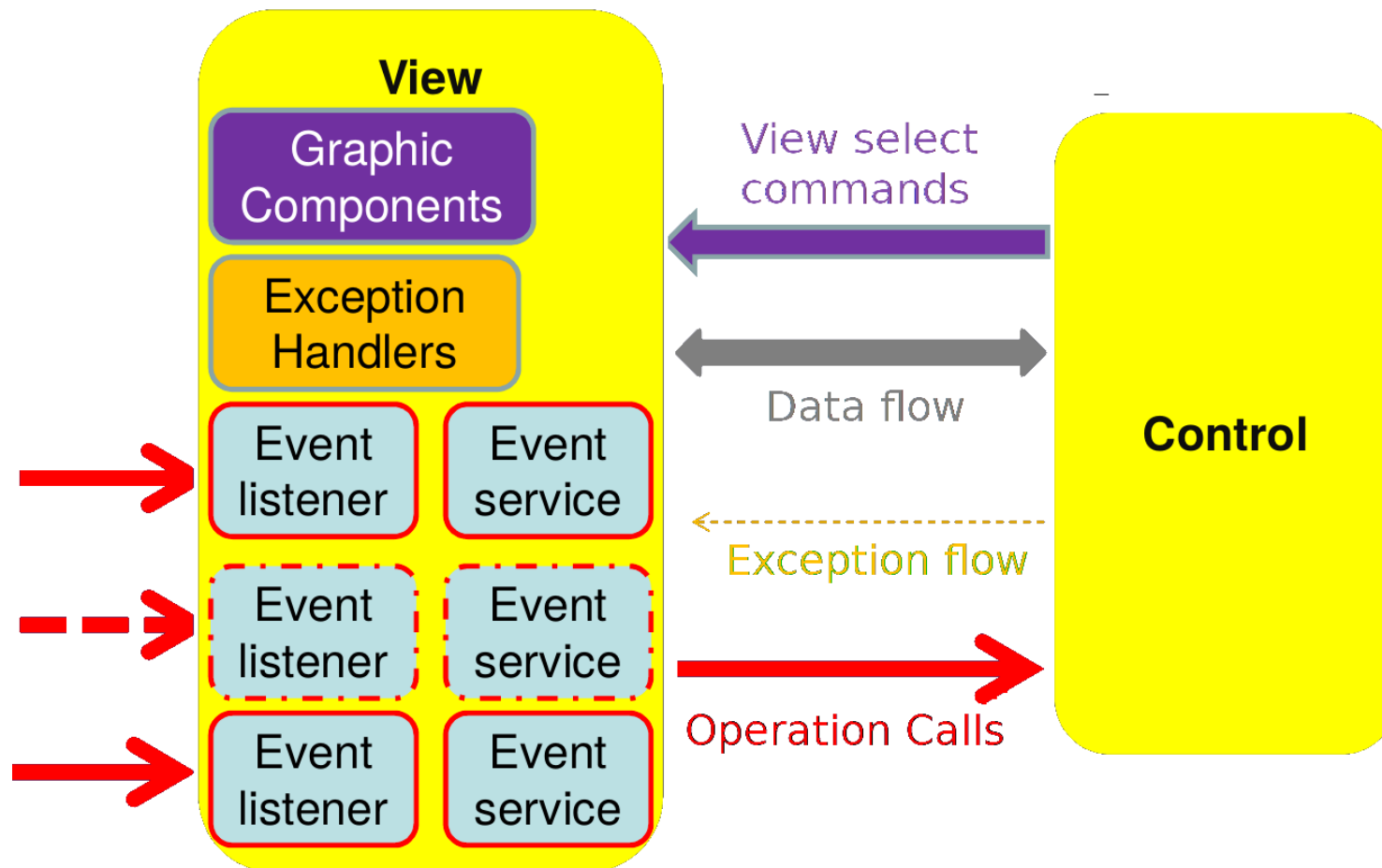
MVC – controller

- controller grafico dell'applicazione: coordina l'interazione tra View ed il Controller Applicativo (o il Model in sola lettura) per la realizzazione di una funzionalità
 - converte formati esterno in interno e viceversa (se non a carico della View)
 - realizza la mappa tra input dell'utente e processi eseguiti dal Model
 - crea/seleziona le istanze di View richieste
- controller applicativo: implementa la logica di controllo dell'applicazione
 - raffina il concetto di **Controller in BCE**
 - - quale soluzione prediligiamo in questo corso?
 - perché?
 - in preparazione dell'esame, argomentare le risposte
 - non dovrebbe esistere uno stato comune tra le istanze
 - attivazioni diverse dello stesso caso d'uso fanno riferimento a diverse istanze di controller applicativo
- progettazione ed implementazione:
 - controller grafico ed applicativo aggregati in un unico “bundle”
 - il controller grafico implementa direttamente la logica di controllo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono anche sul controller
 - controller grafico ed applicativo disaccoppiati
 - il controller grafico viene associato a parte della view
 - il controller grafico invoca operazioni sul controller applicativo
 - se la view evolve, l'impatto dei cambiamenti si ripercuotono solo sul controller applicativo e non sul modo in cui il caso d'uso è realizzato (i.e. controller applicativo)

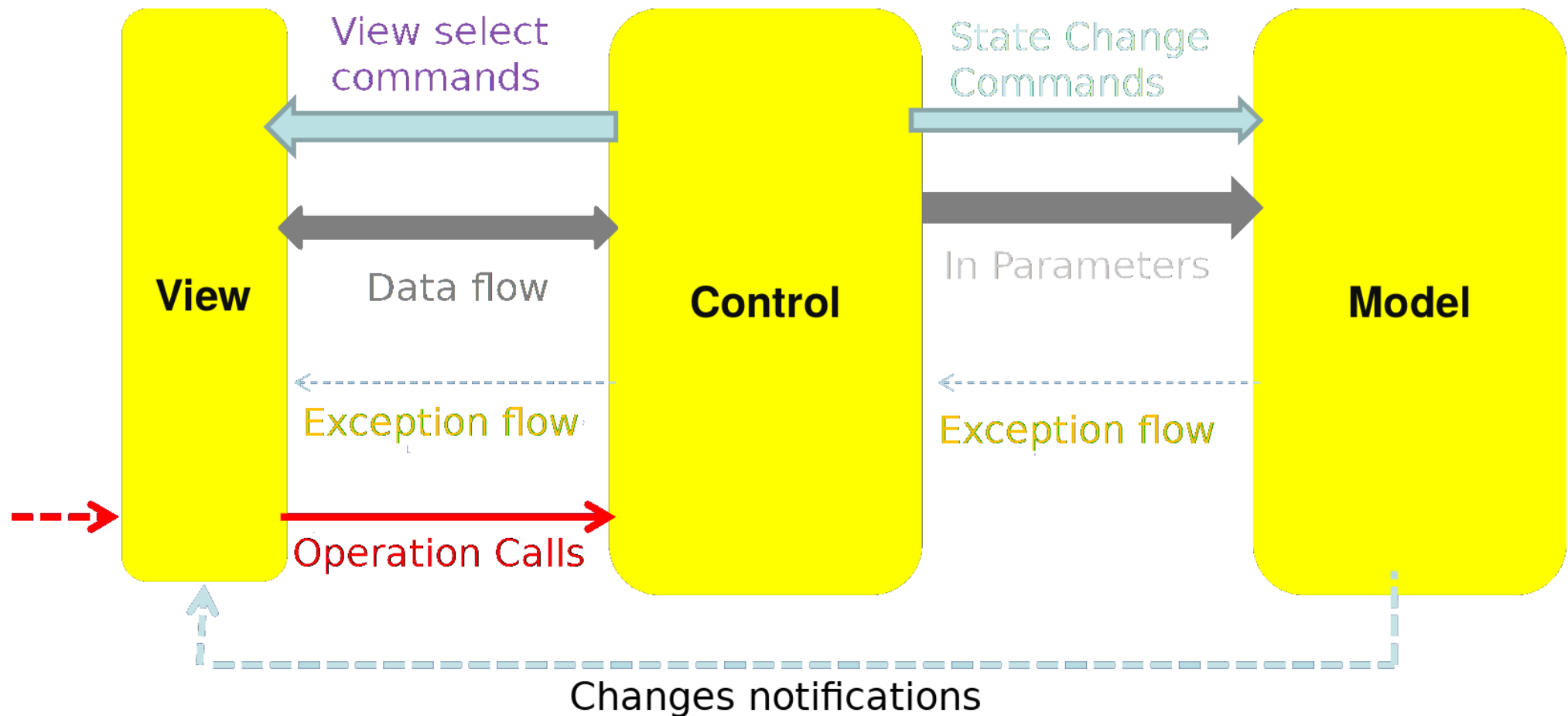
MVC – model

- costituisce la rete di entità partecipanti alla logica di applicazione
 - per ogni entità
 - descrive i comportamenti esposti in forma di operazioni
 - gestisce/realizza le relazioni tra le entità
- come nel caso Entity di BCE, anche per il Model MVC
 - incapsulare lo stato (i.e. attributi) delle entità dell'applicazione enfatizzando il ruolo delle operazioni offerte
 - un buon modello di entità non dovrebbe esporre direttamente la rappresentazione dello stato di un oggetto
 - non dipendere dalla specifica rappresentazione degli attributi
 - enfatizzare il ruolo delle operazioni non limitarle a getter/setter
- può avere la responsabilità di notificare ai componenti View eventuali aggiornamenti verificatisi (in seguito a richieste del Controller)

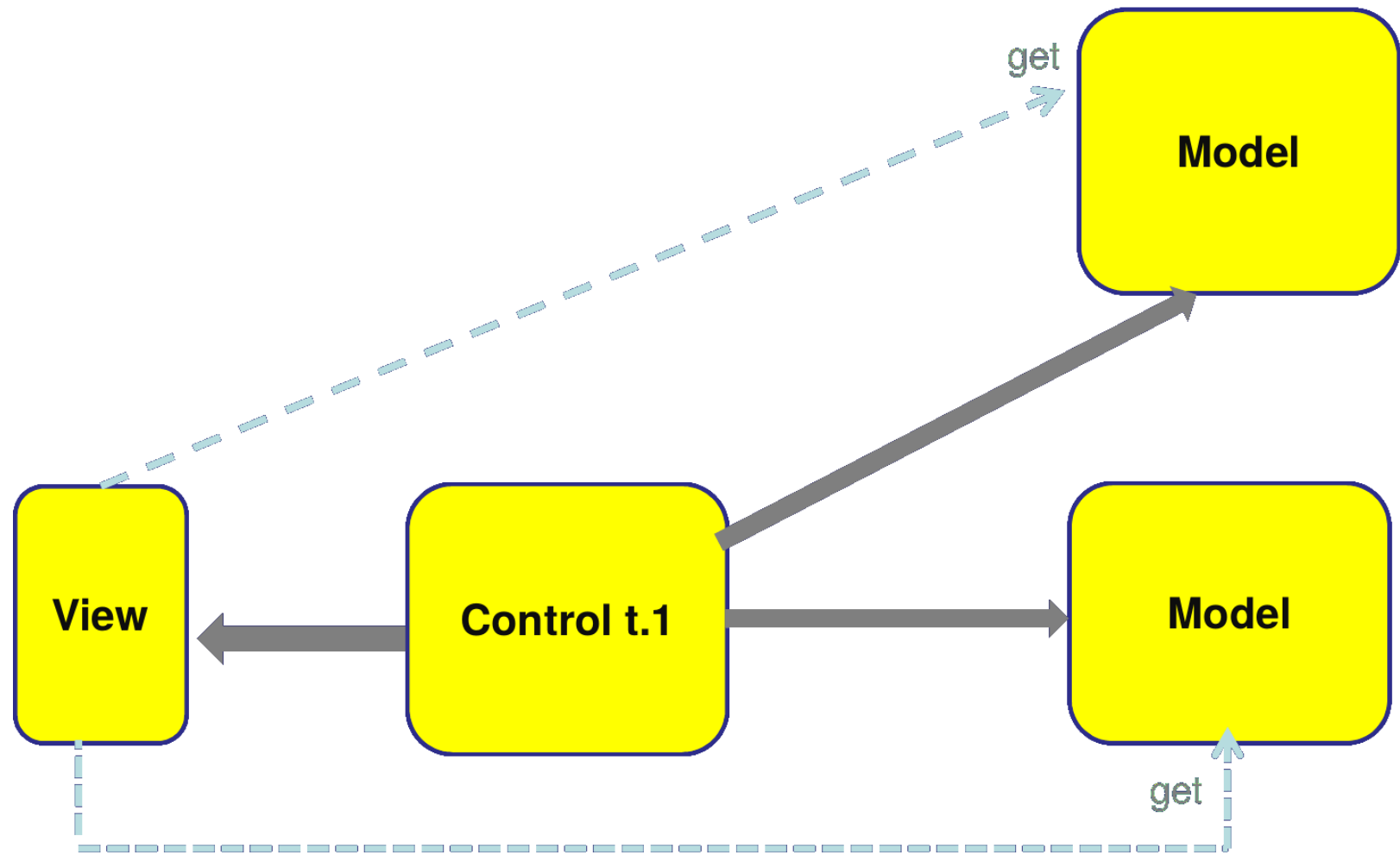
MVC – interazioni utente



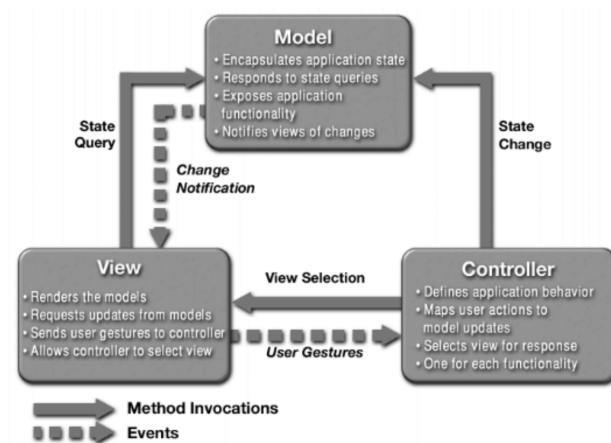
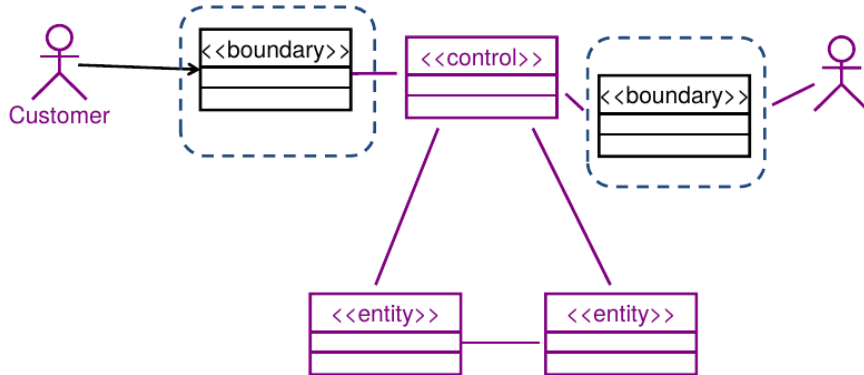
MVC – interazioni utente && push-model



MVC – interazioni utente && pull-model



gestione delle evoluzioni in BCE e MVC

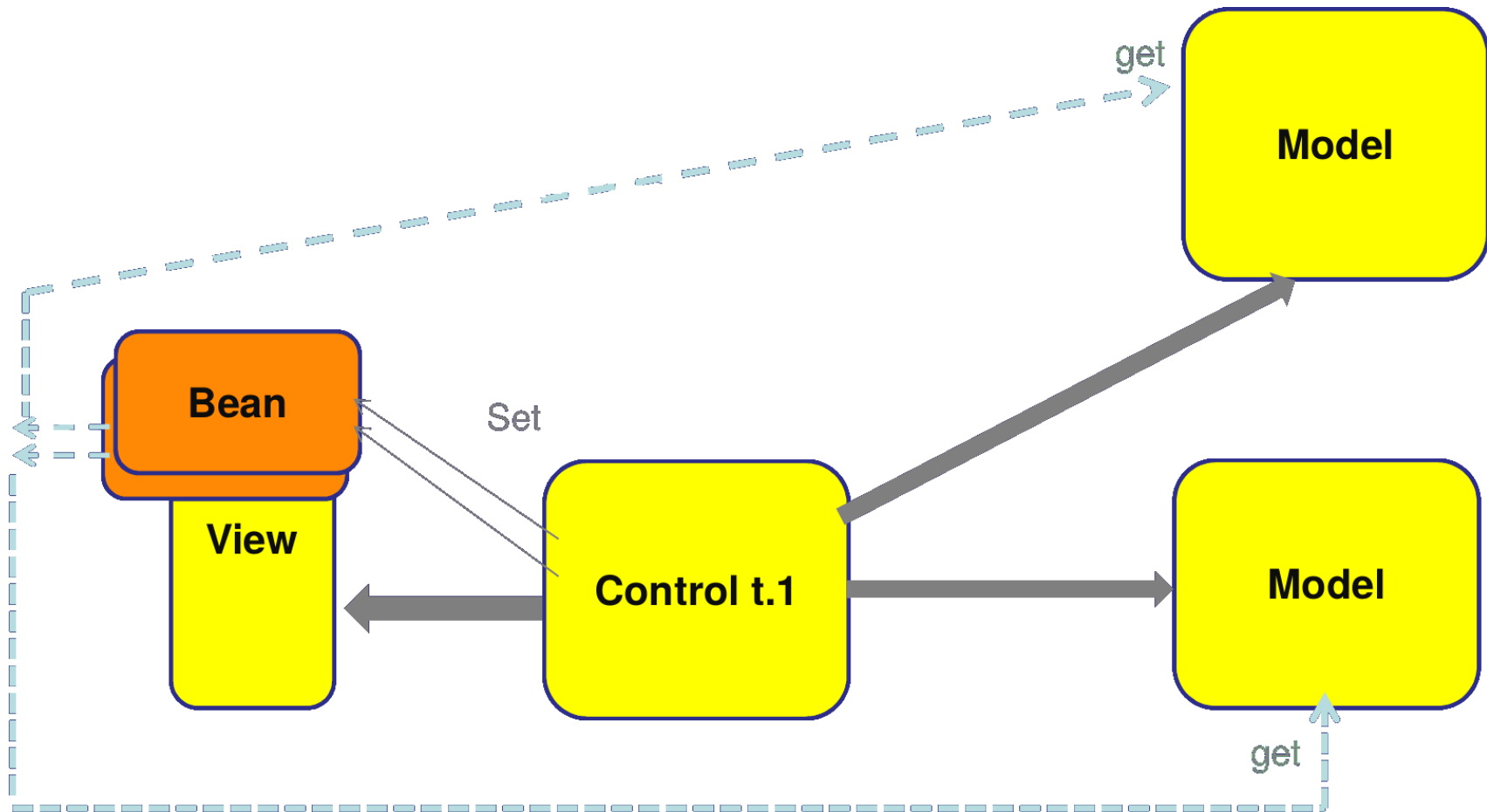


- evoluzione del Model → quali impatti a seguito di:
 - evoluzione dei formati di rappresentazione delle informazioni
 - evoluzione delle key abstractions di una applicazione
 - evoluzione della rappresentazione delle key abstractions di una applicazione
 - riorganizzazione delle relazioni tra key abstractions
- evoluzioni sulla View/Boundary → quali impatti nel caso di:
 - evoluzioni strettamente legate alla variazione dei meccanismi di IO
 - gestione simultanea di più versioni/varianti della stessa View

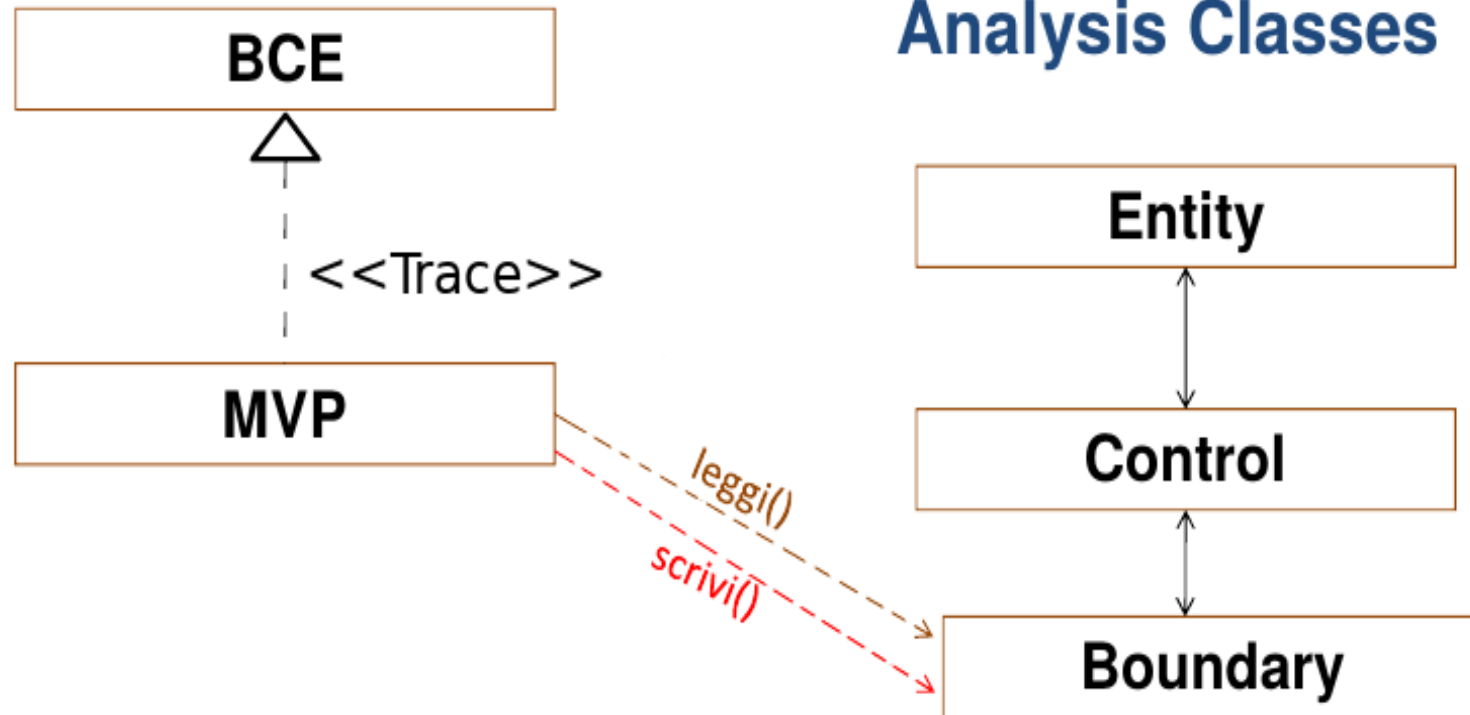
classi “bean”

- in generale:
 - elemento di buffering per la comunicazione fra classi Boundary, Control ed Entity
 - principalmente sono interposte fra le classi Boundary e il resto delle classi del VOPC
 - **MODELLARE LE CLASSI BEAN FIN DAL BCE, PER POI RAFFINARLE IN MVC**
- principali responsabilità
 - disaccoppiamento tra classi Boundary ed Entity
 - controllo della validazione sintattica sull’inserimento dei dati in input
 - eventuale gestione dei riferimenti per i dati in visualizzazione
 - gestione pull-mode o push-mode
- struttura tipica
 - realizzano un POJO (Plain Old Java Object)
 - attributi privati destinati a contenere dati di IO
 - principalmente metodi pubblici solo di tipo setter e getter
 - accesso agli attributi solo attraverso IO setter e getter
 - metodi privati per il controllo sintattico dei dati

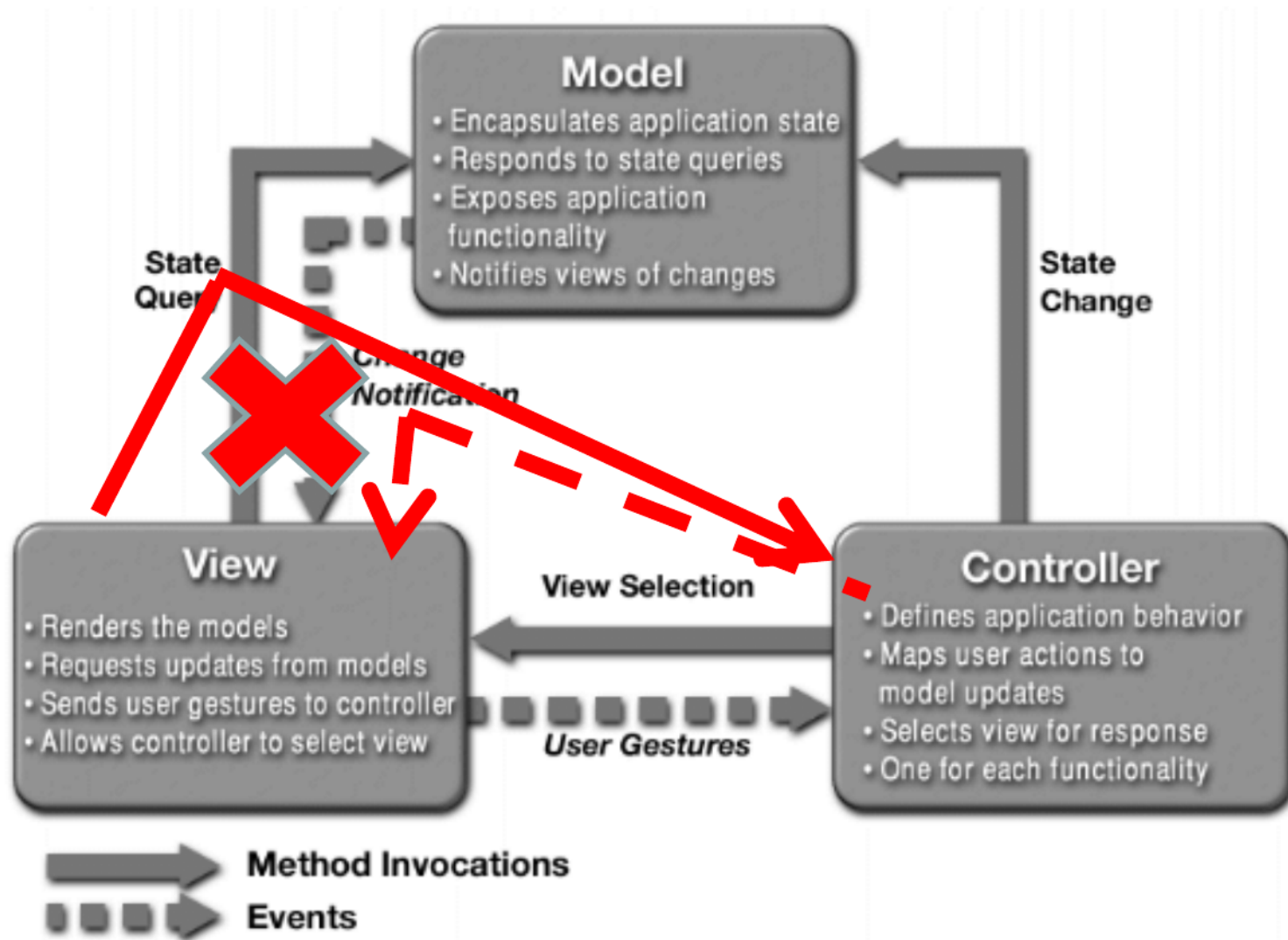
MVC – pull-model con beans



model-view-presenter



model-view-presenter



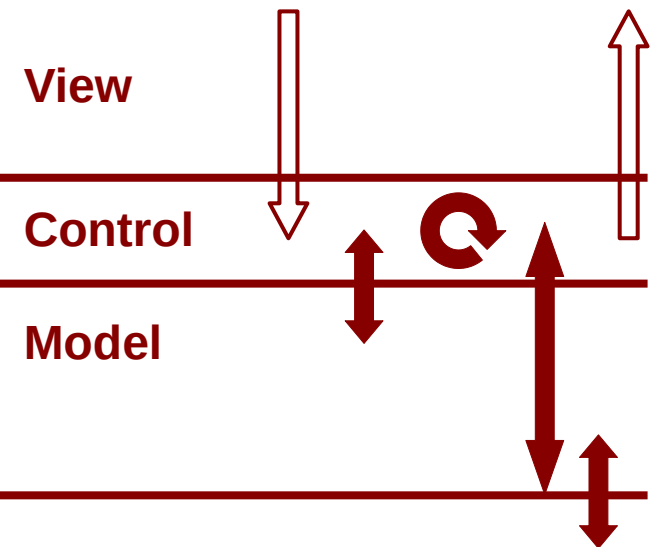
MVP – dettagli

- View
 - i dati da presentare **NON POSSONO** essere acquisiti direttamente dal Model
 - la View ha accesso ai metodi del Model del tipo Attributo (e.g. `getAttributo()`)
 - la View deve sempre interagire con il Controller
 - le restanti responsabilità sono pari a quelle di View di MCV
 - anche nel caso MVP i dati scambiati tra la parte View e la parte Presentation vanno intesi come classi Bean di appoggio
- Presentation Model
 - come Control di MVC.
- Model
 - come per MVC
- principalmente utilizzato per sviluppare GUI

MVC in architetture laptop/stand-alone

- tutti i tre gli strati risiedono sulla macchina “utente”
- struttura tipica:

- Grafica, listener, controllore UI e relazione con controllore di UC
- Classi di controllo (controller applicativi)
- Classi entità e di relativa interfaccia verso la persistenza
- Persistenza



- Interazioni lettura/scrittura mediate da classi bean
- Interazioni lettura/scrittura SENZA la mediazione di classi bean

in architetture laptop/stand-alone ...

- tipicamente sono scenari altamente centralizzati
 - l'esecuzione dell'applicazione risiede (per lo più) in un'unica istanza di JVM
 - tecnologie di programmazione omogenee, spesso univoche
- le interazioni tra le istanze avvengono in memoria
 - tutte le istanze condividono lo stesso spazio di indirizzamento
 - i riferimenti tra le istanze di fatto sono riferimenti ad indirizzi di memoria
 - lo scambio di informazioni tra istanze è principalmente orientato allo scambio di tali riferimenti

in architetture distribuite ...

- scenari che coinvolgono distinte unità computazionali
 - l'esecuzione dell'applicazione risiede su più istanze di JVM, tipicamente in esecuzione su macchine fisiche differenti
 - tecnologie di programmazione non necessariamente omogenee
- le interazioni tra le istanze non possono avvenire direttamente in memoria
 - in generale non è previsto uno spazio di memoria condiviso
 - ogni unità dell'applicazione distribuita ha un proprio spazio di indirizzamento, fisicamente separato dagli altri (o perché è su un'altra macchina, o perché è il SO che garantisce per la separazione)
 - i riferimenti tra le istanze corrispondono ad indirizzi di memoria solo all'interno della stessa JVM. Dati condivisi tra più JVM devono essere duplicati in più istanze differenti, che (per quanto possibile) vanno mantenuti consistenti
 - lo scambio di informazioni tra istanze è principalmente orientato allo scambio di copie di istanze
 - in formato binario, se le architetture distribuite usano soluzioni omogenee
 - in rappresentazioni portabili, se le architetture distribuite usano soluzioni non omogenee (e.g. linguaggi di programmazioni diversi)

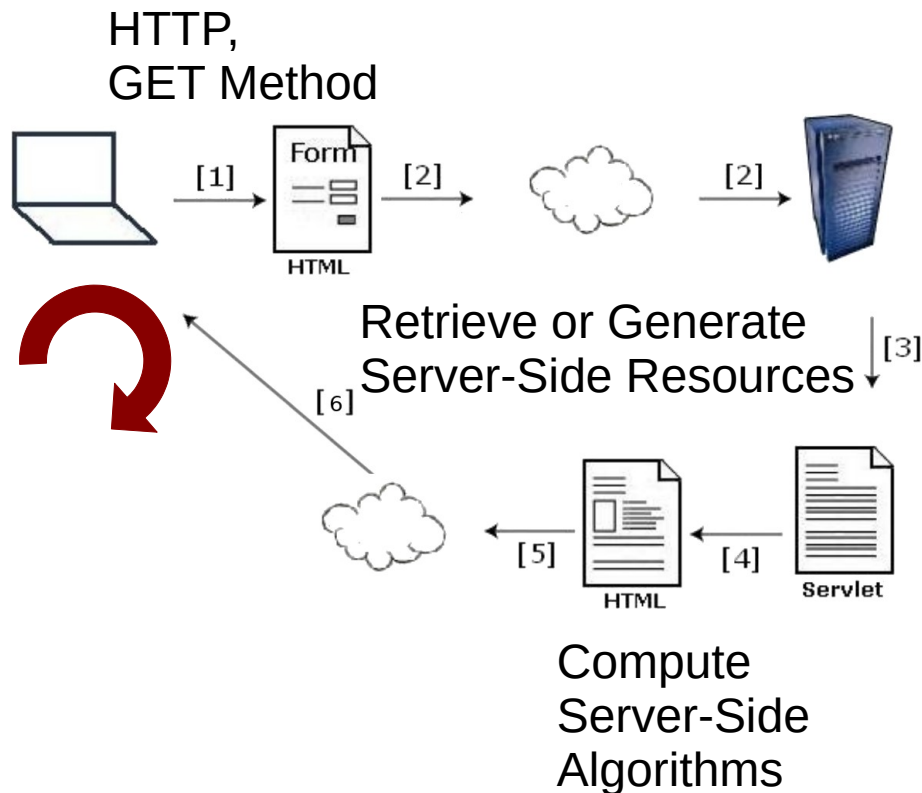
in architetture distribuite ...

- nelle architetture centralizzate l'uso di classi BEAN INCORAGGIA una progettazione che separi la logica di organizzazione dei dati NEL sistema dalla loro presentazione/utilizzo agli attori DEL sistema
- nelle architetture distribuite diventa NECESSARIA l'introduzione di classi di supporto che mitigano i problemi di comunicazione tra:
 - le classi responsabili dell'implementazione dei casi d'uso (i.e. CONTROLLER applicativi)
 - le classi responsabili dell'interazione degli attori nei casi d'uso (i.e. BOUNDARY e loro raffinamento in VIEW)

separazione)

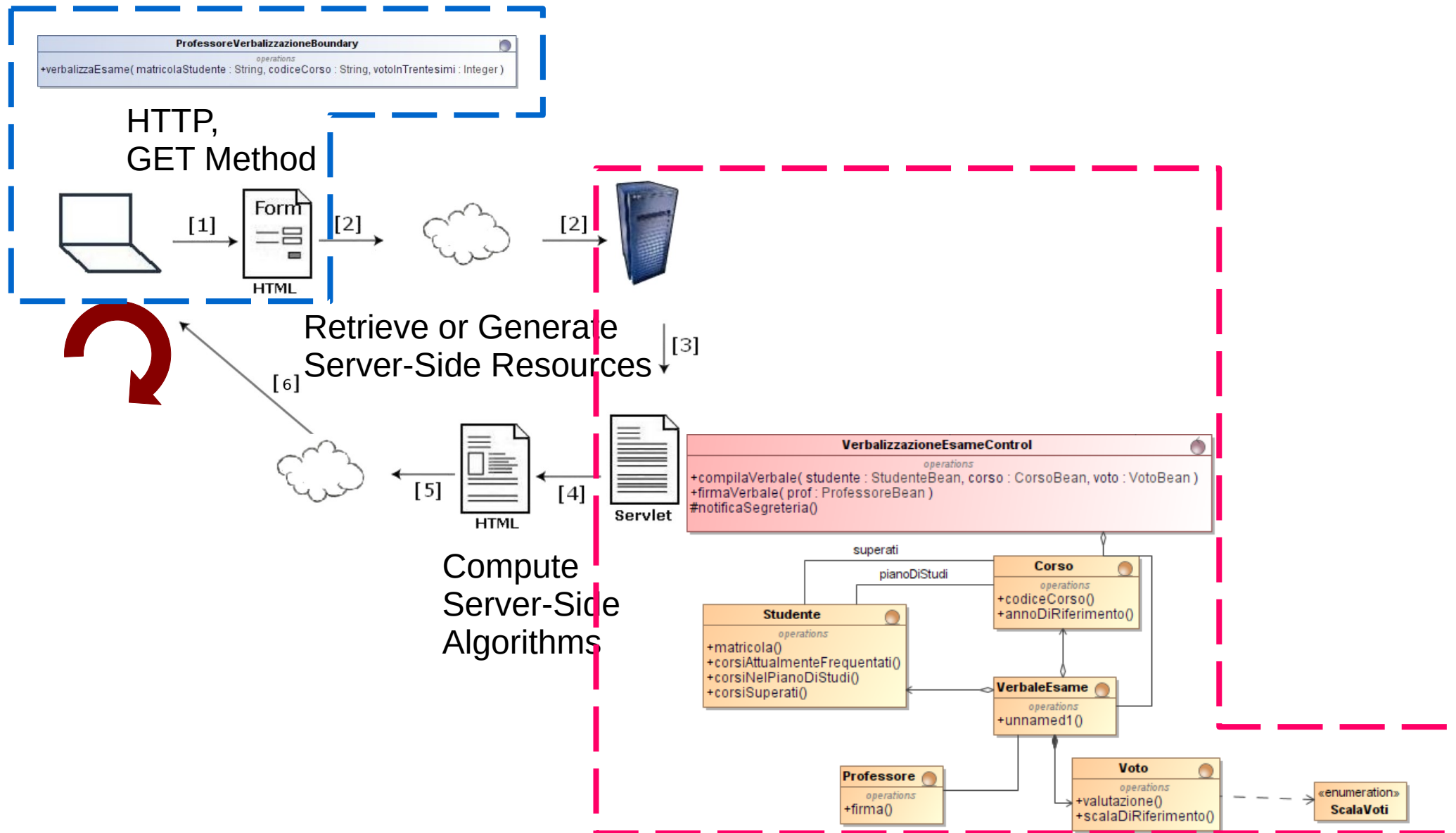
- i riferimenti tra le istanze corrispondono ad indirizzi di memoria solo all'interno della stessa JVM. Dati condivisi tra più JVM devono essere duplicati in più istanze differenti, che (per quanto possibile) vanno mantenuti consistenti
- lo scambio di informazioni tra istanze è principalmente orientato allo scambio di copie di istanze
 - in formato binario, se le architetture distribuite usano soluzioni omogenee
 - in rappresentazioni portabili, se le architetture distribuite usano soluzioni non omogenee (e.g. linguaggi di programmazioni diversi)

architetture distribuite – contesto generale (web)

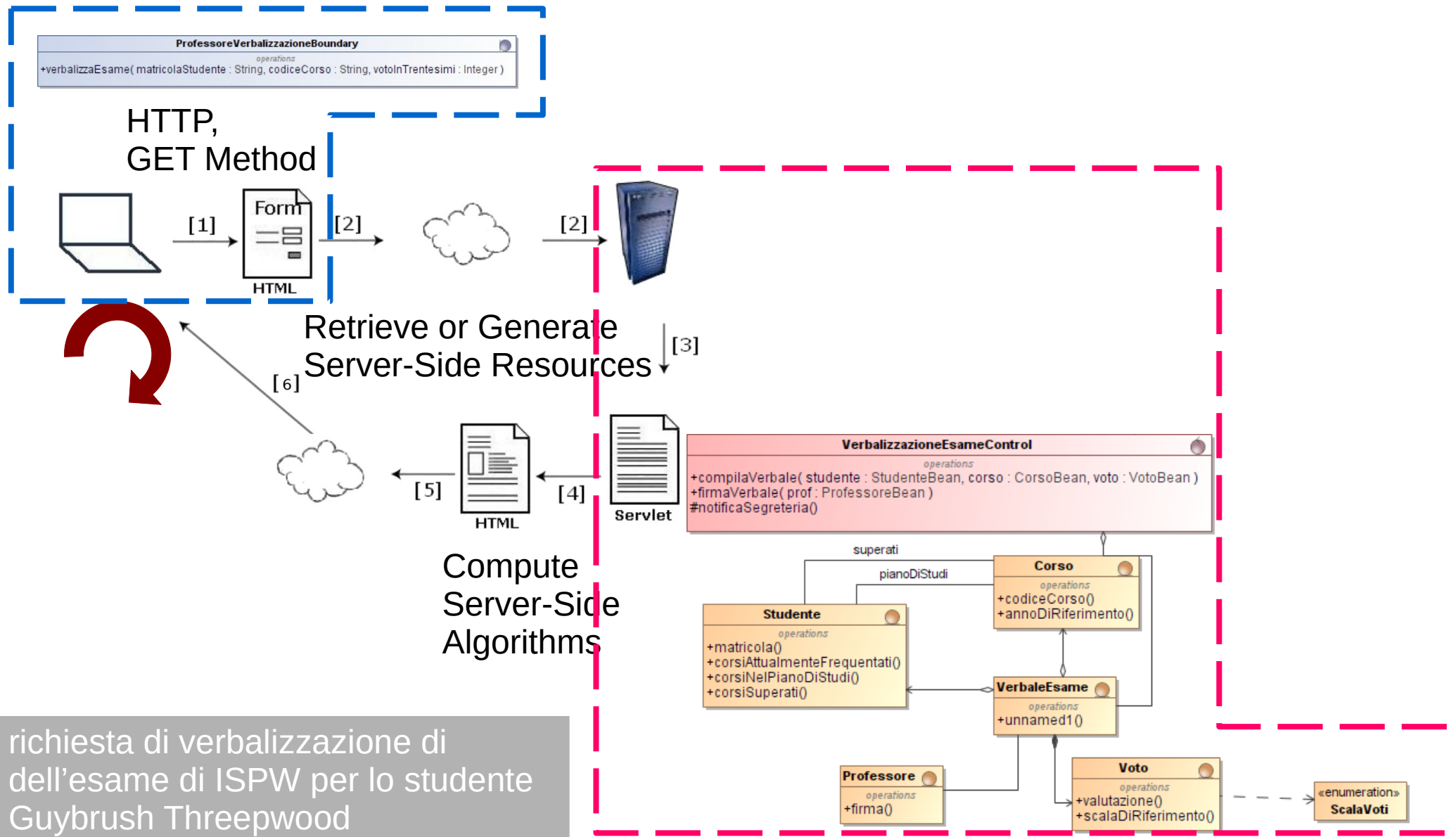


- applicazioni distribuite che forniscono funzionalità accessibili da remoto utilizzando “thin-clients”
 - web browser
- la View non è installata sulle macchine “utente”
 - la View ed almeno parte del Controller grafico (Applet Java, oppure HTML + Java Scripts, o altro) sono generati server-side e trasferiti sul browser del client per l'esecuzione
 - impossibilità di trasferire istanze di entity e loro riferimenti
 - è necessario (non solo utile) introdurre classi di appoggio per la gestione si IO → classi Bean
- il controller ed il model (versione O.O.) risiedono server-side

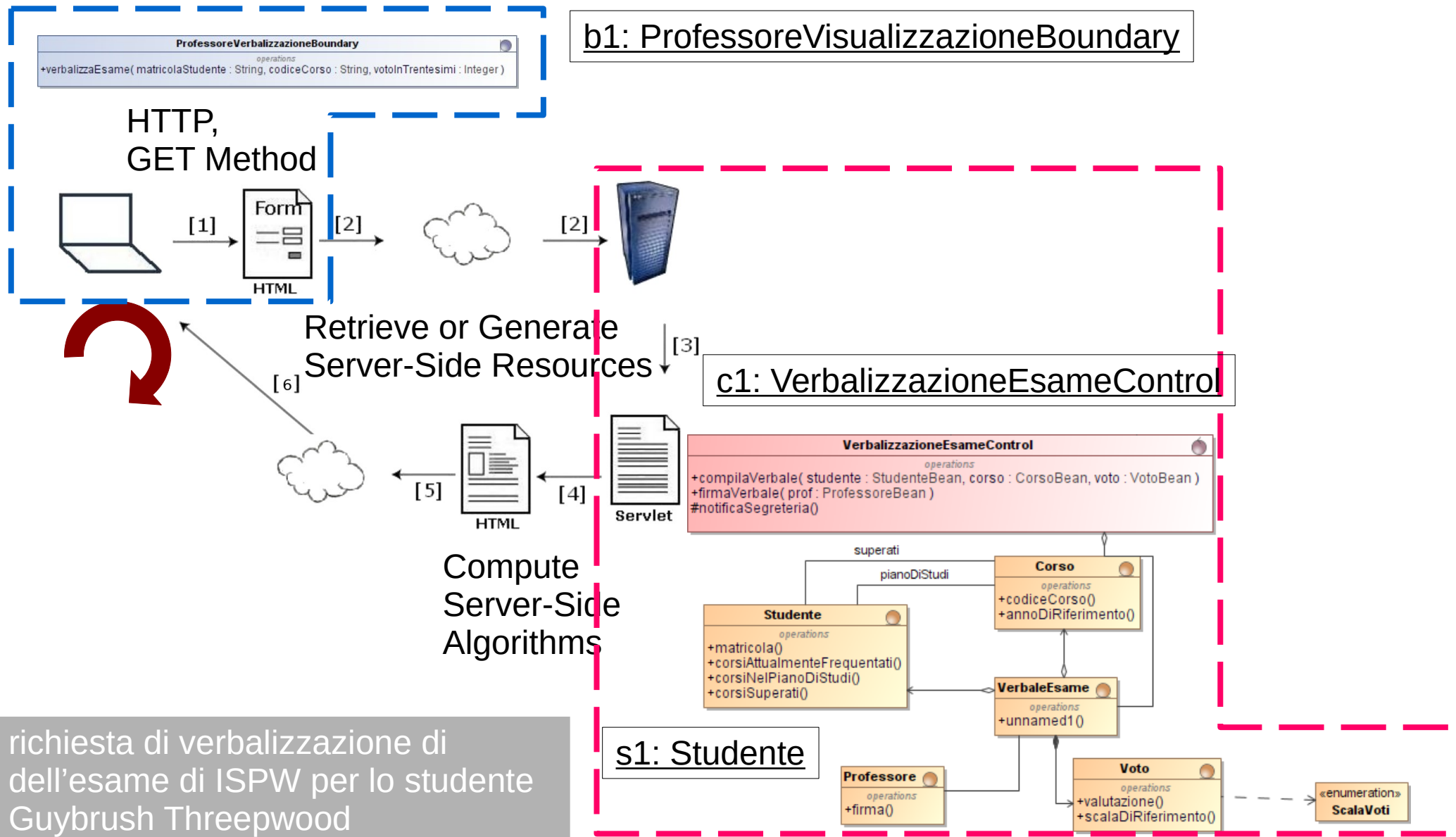
architetture distribuite – allocazione delle istanze



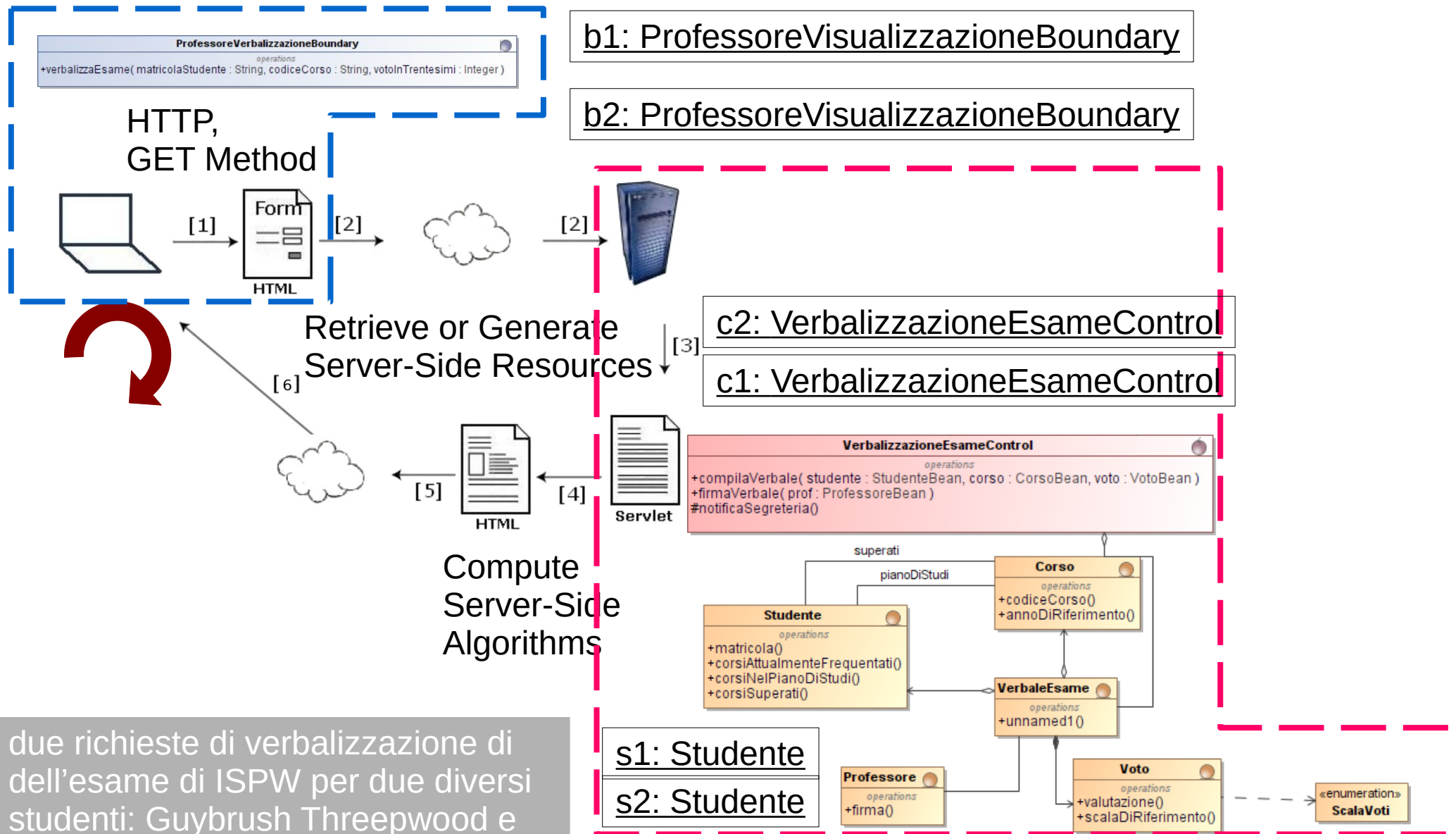
architetture distribuite – allocazione delle istanze



architetture distribuite – allocazione delle istanze

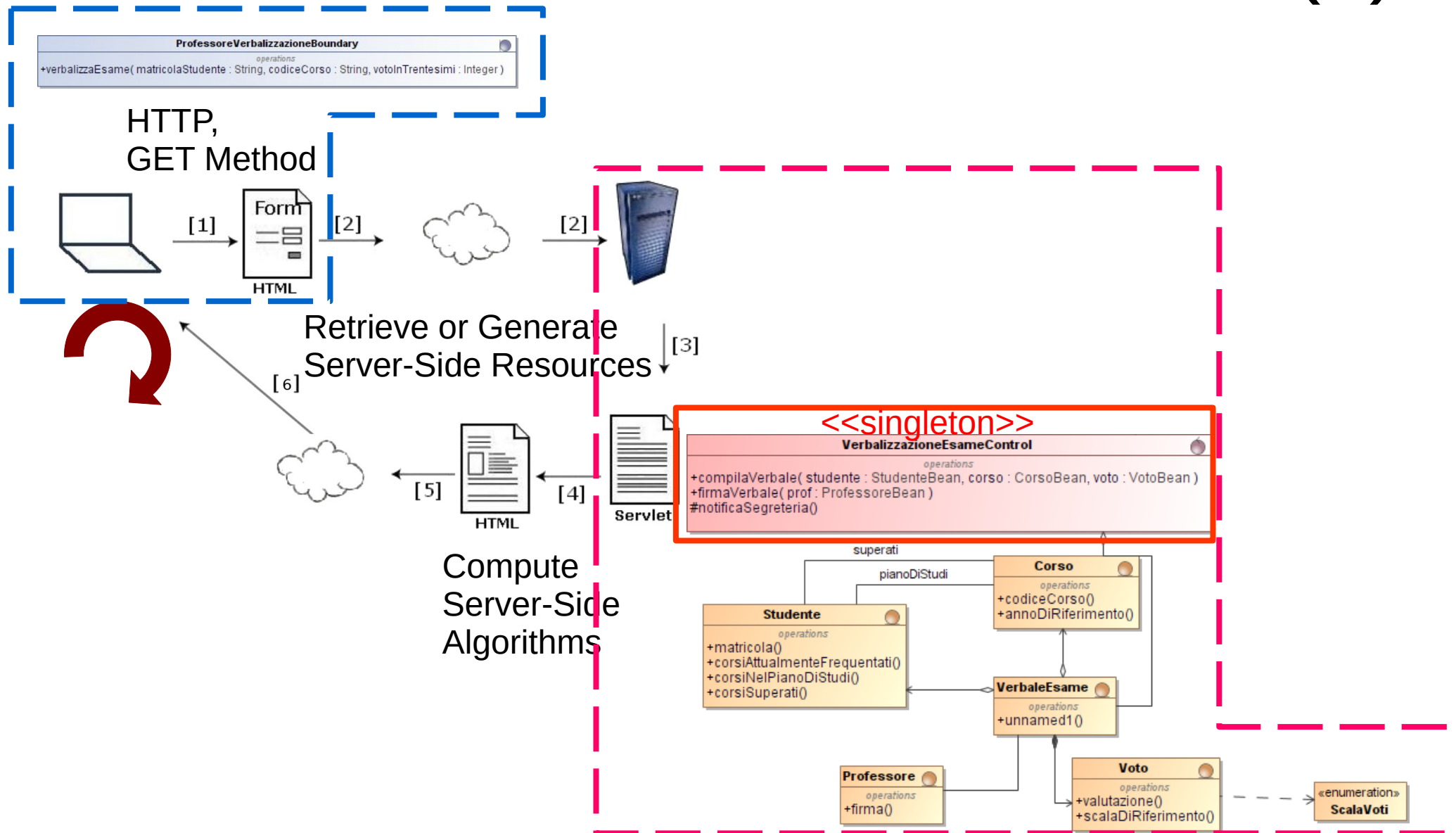


architetture distribuite – allocazione delle istanze

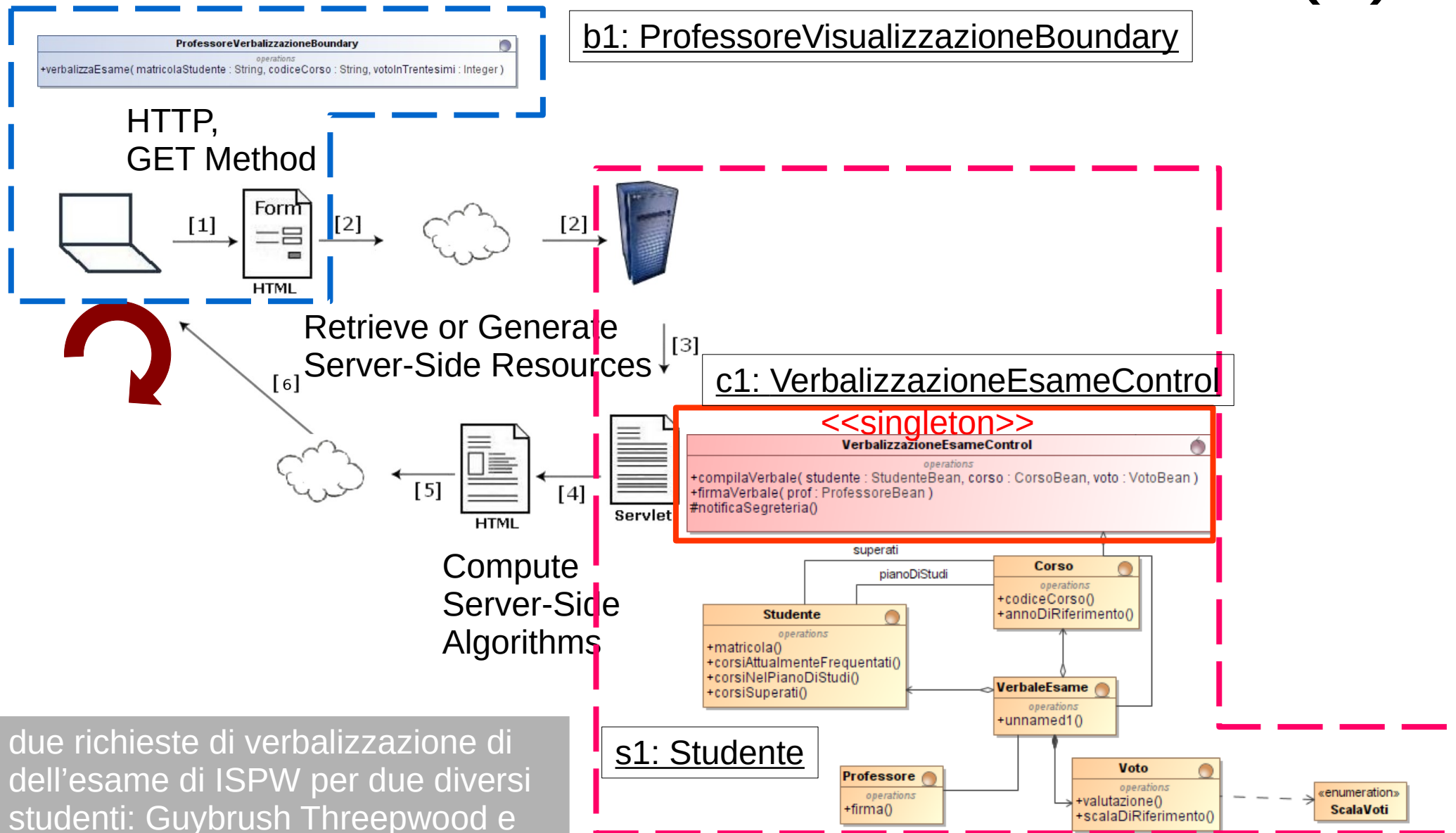


due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

architetture distribuite – attenzione!!! (1)

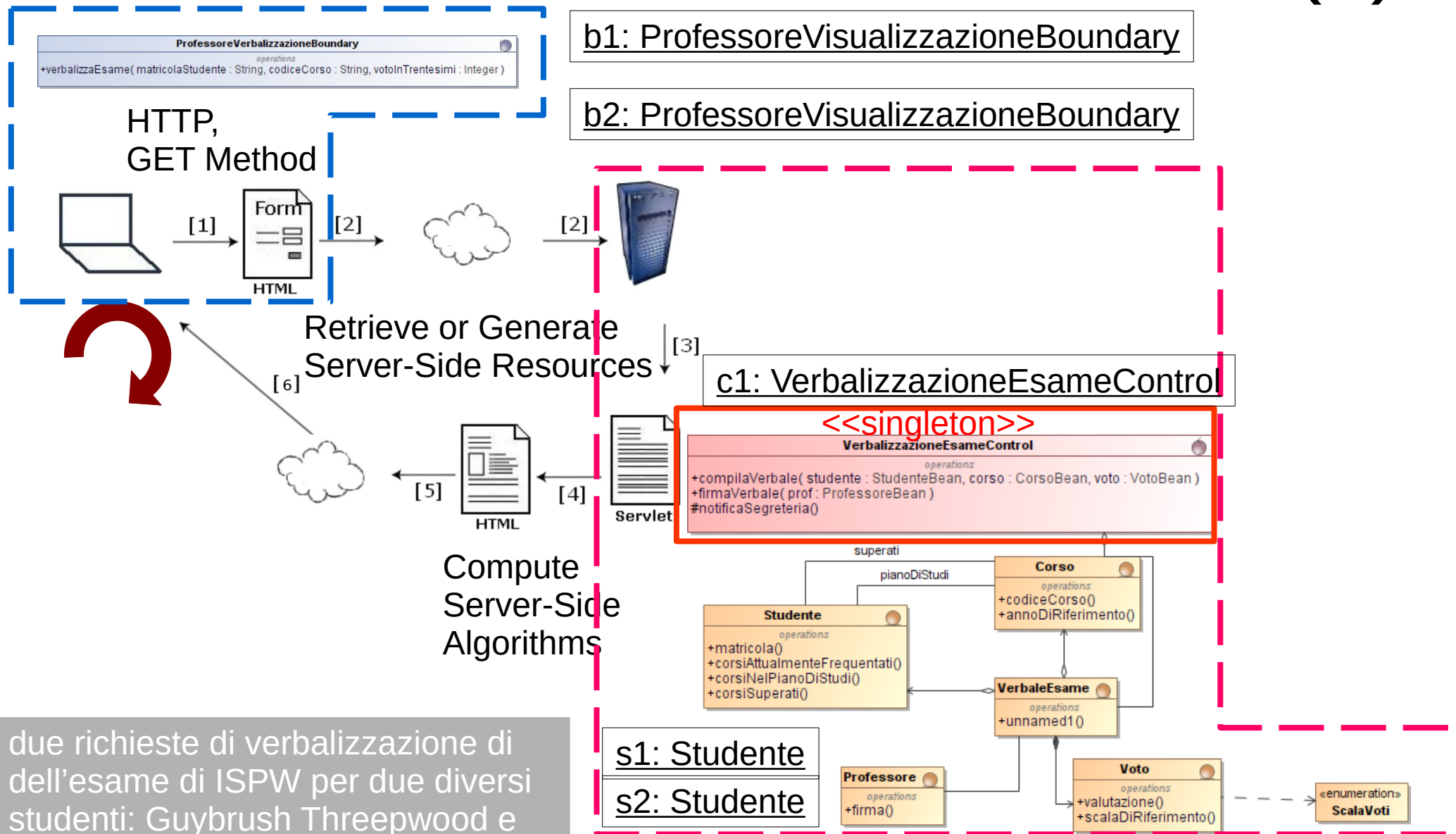


architetture distribuite – attenzione!!! (1)



due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

architetture distribuite – attenzione!!! (1)



due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

architetture distribuite – attenzione!!! (1)

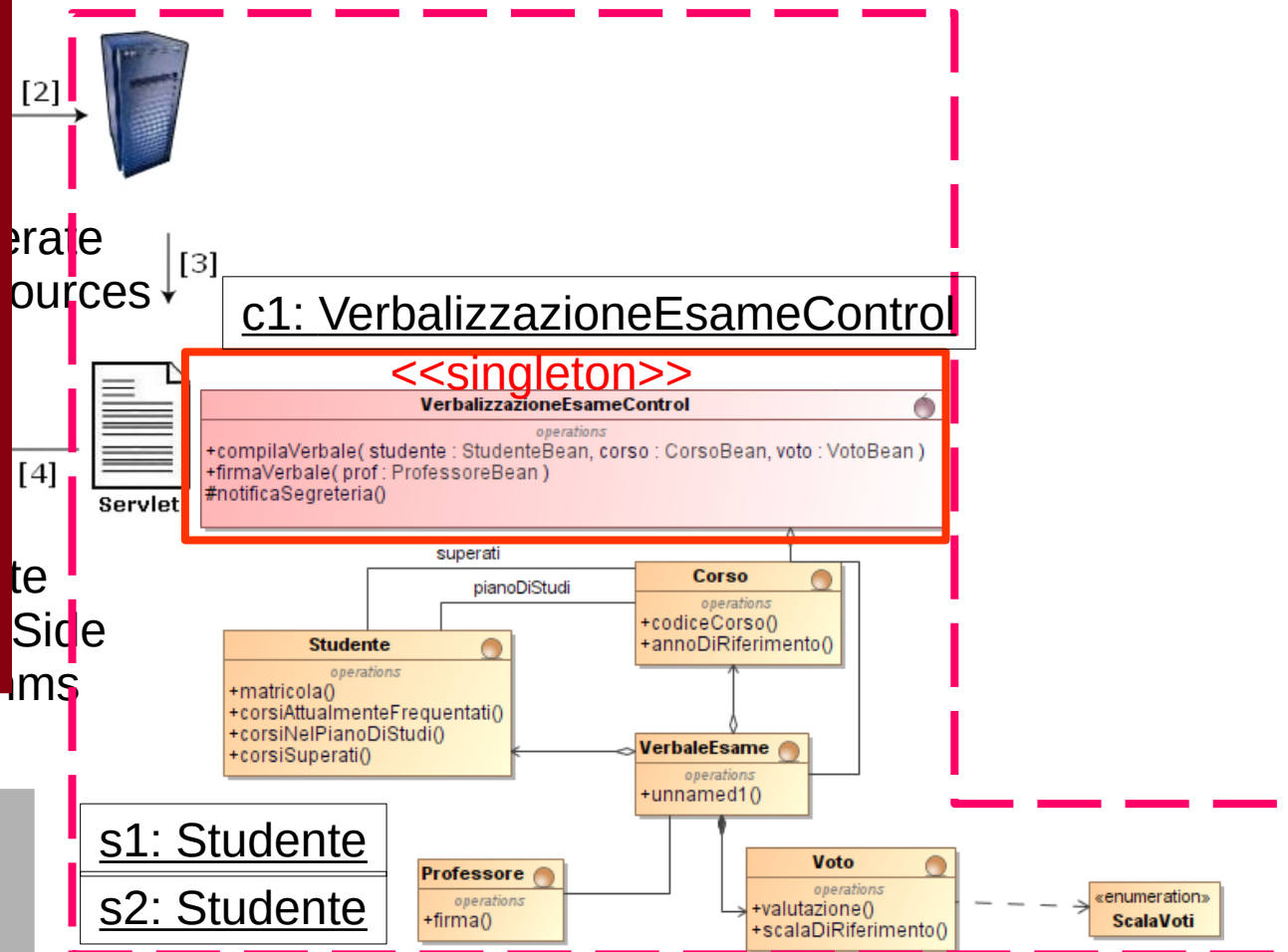
ProfessoreVerbalizzazioneBoundary
operations
+verbalizzaEsame(matricolaStudente : String, codiceCorso : String, votoInTrentesimi : Integer)

Rischioso:

- tutte le richieste condividono l'unica istanza di controller applicativo
- abbastanza plausibilmente si potranno manifestare errori dovuti all'interleaving di richieste diverse
 - nella gestione della verbalizzazione di Elaine Marley lo stato del controller applicativo fa erroneamente riferimento all'istanza studente di Guybrush Threepwood

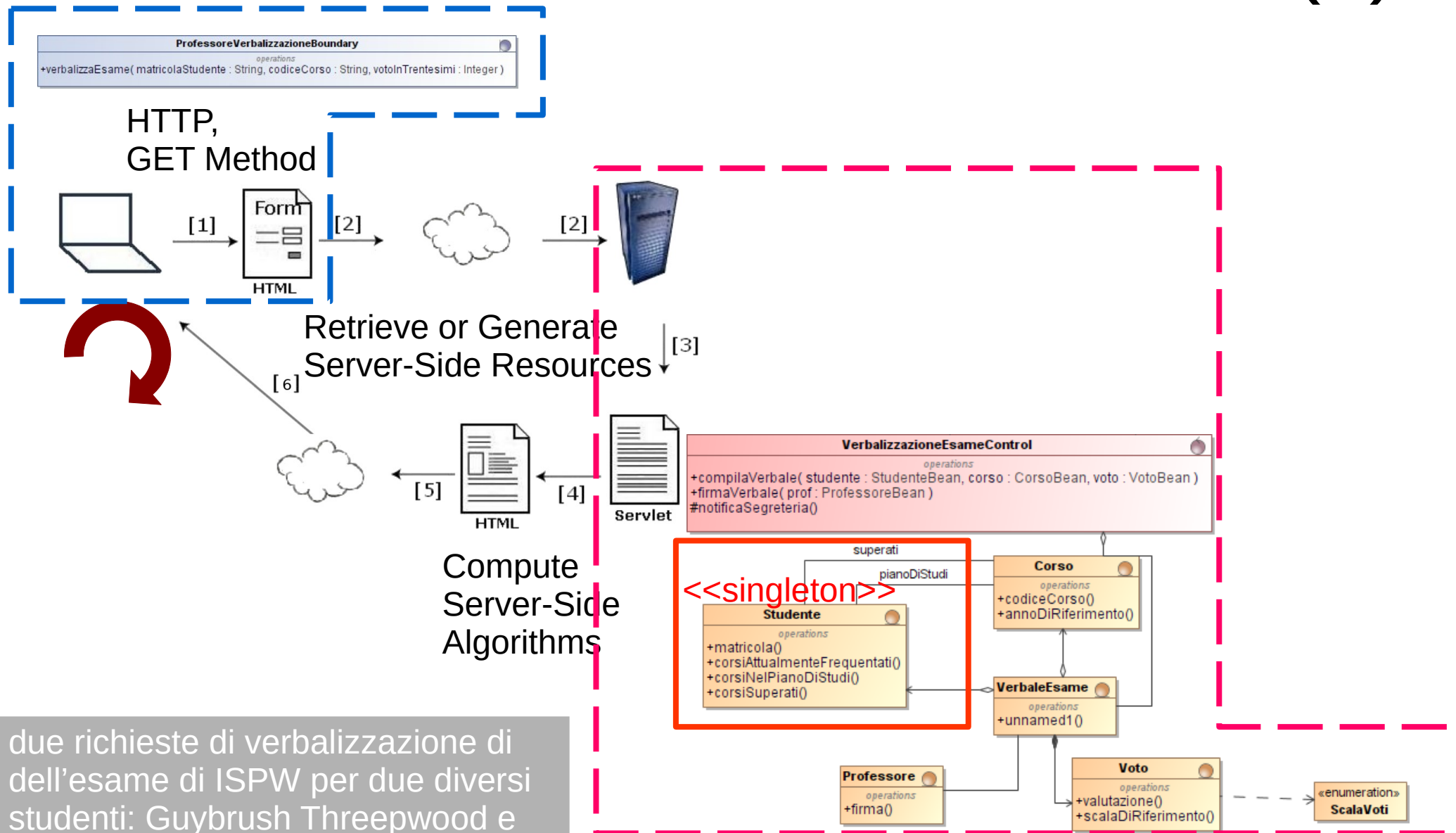
b1: ProfessoreVisualizzazioneBoundary

b2: ProfessoreVisualizzazioneBoundary



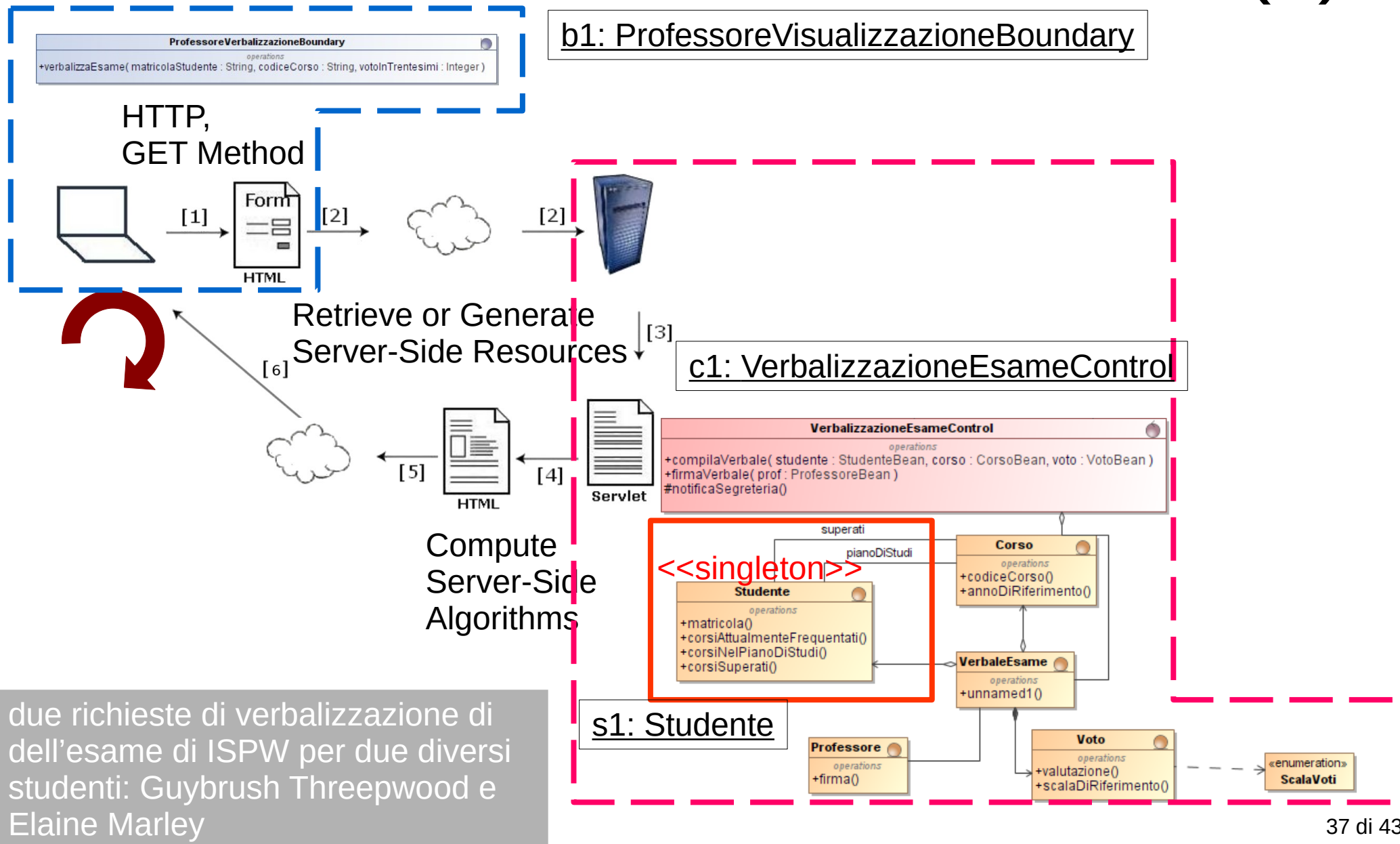
due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

architetture distribuite – attenzione!!! (2)

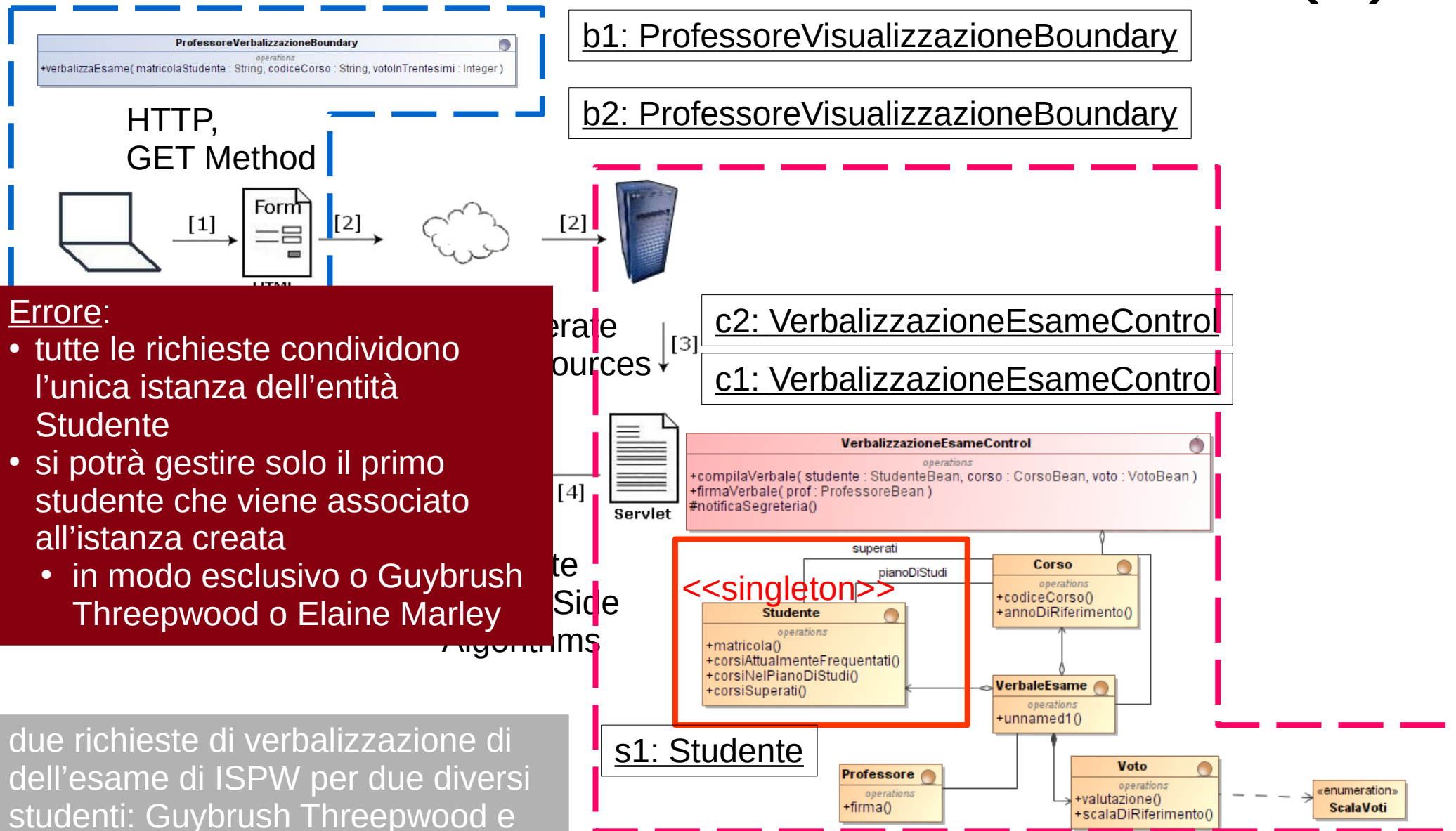


due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

architetture distribuite – attenzione!!! (2)



architetture distribuite – attenzione!!! (2)

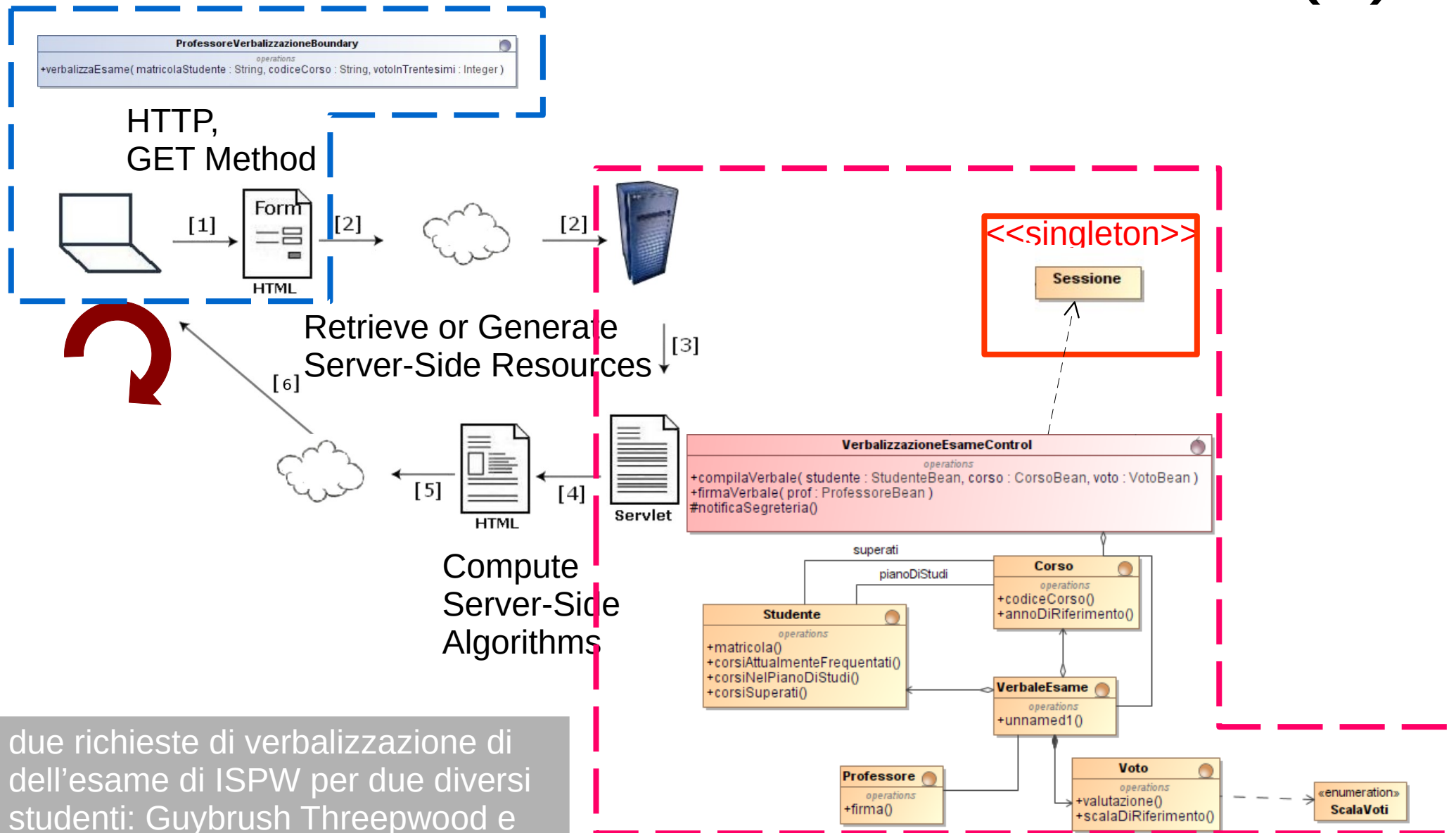


Errore:

- tutte le richieste condividono l'unica istanza dell'entità **Studente**
- si potrà gestire solo il primo studente che viene associato all'istanza creata
 - in modo esclusivo o Guybrush Threepwood o Elaine Marley

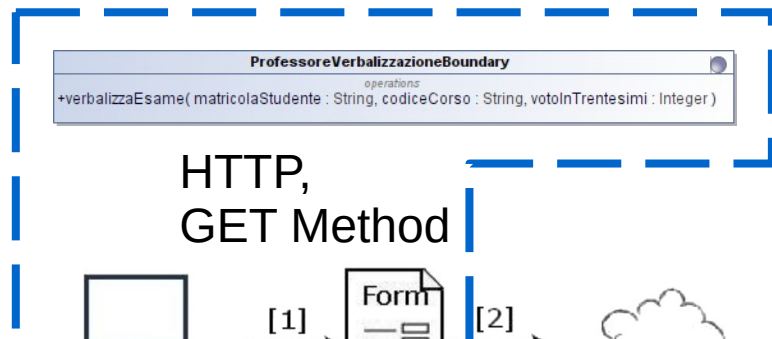
due richieste di verbalizzazione di dell'esame di ISPW per due diversi studenti: Guybrush Threepwood e Elaine Marley

architetture distribuite – attenzione!!! (3)



due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley

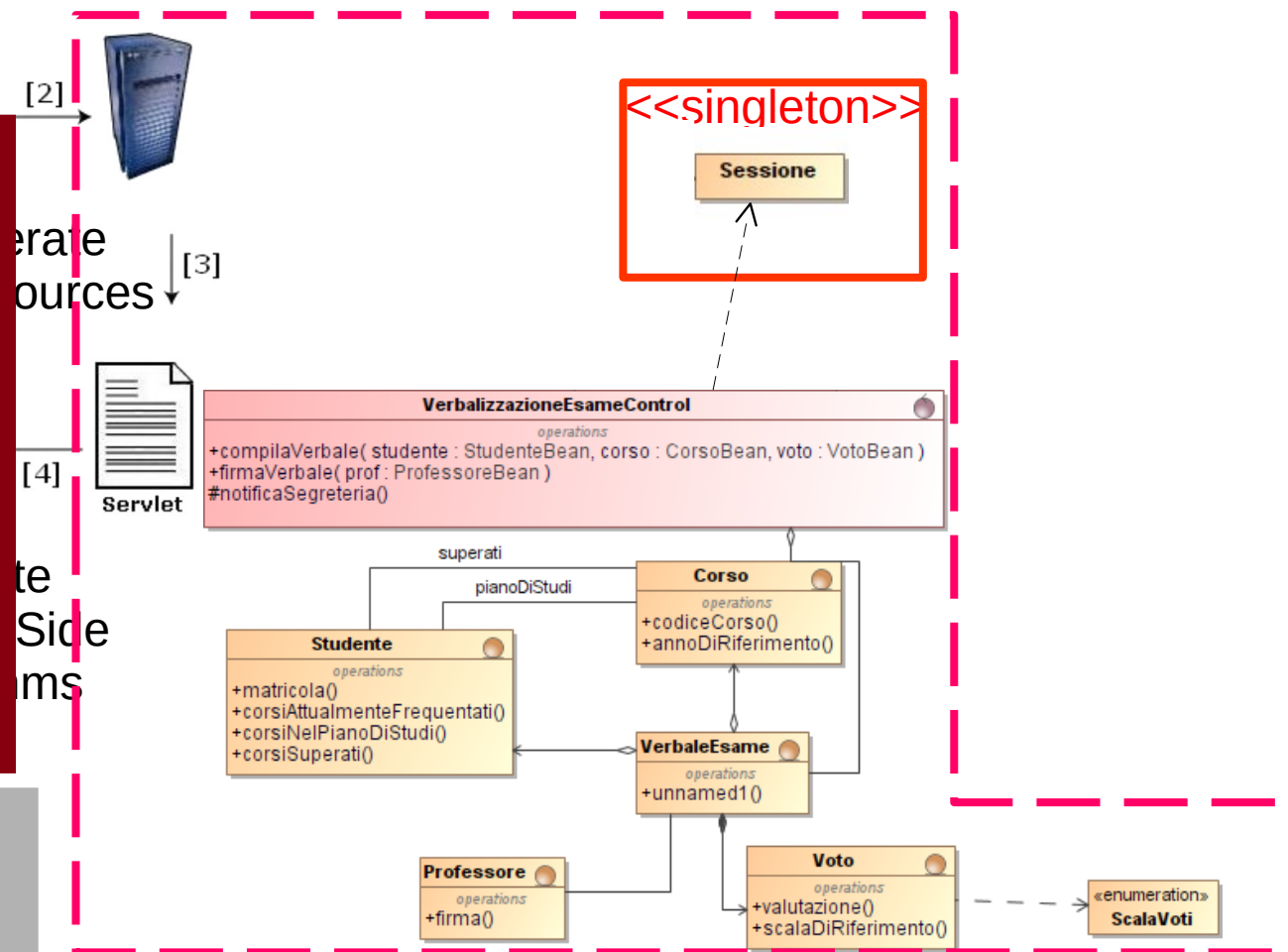
architetture distribuite – attenzione!!! (3)



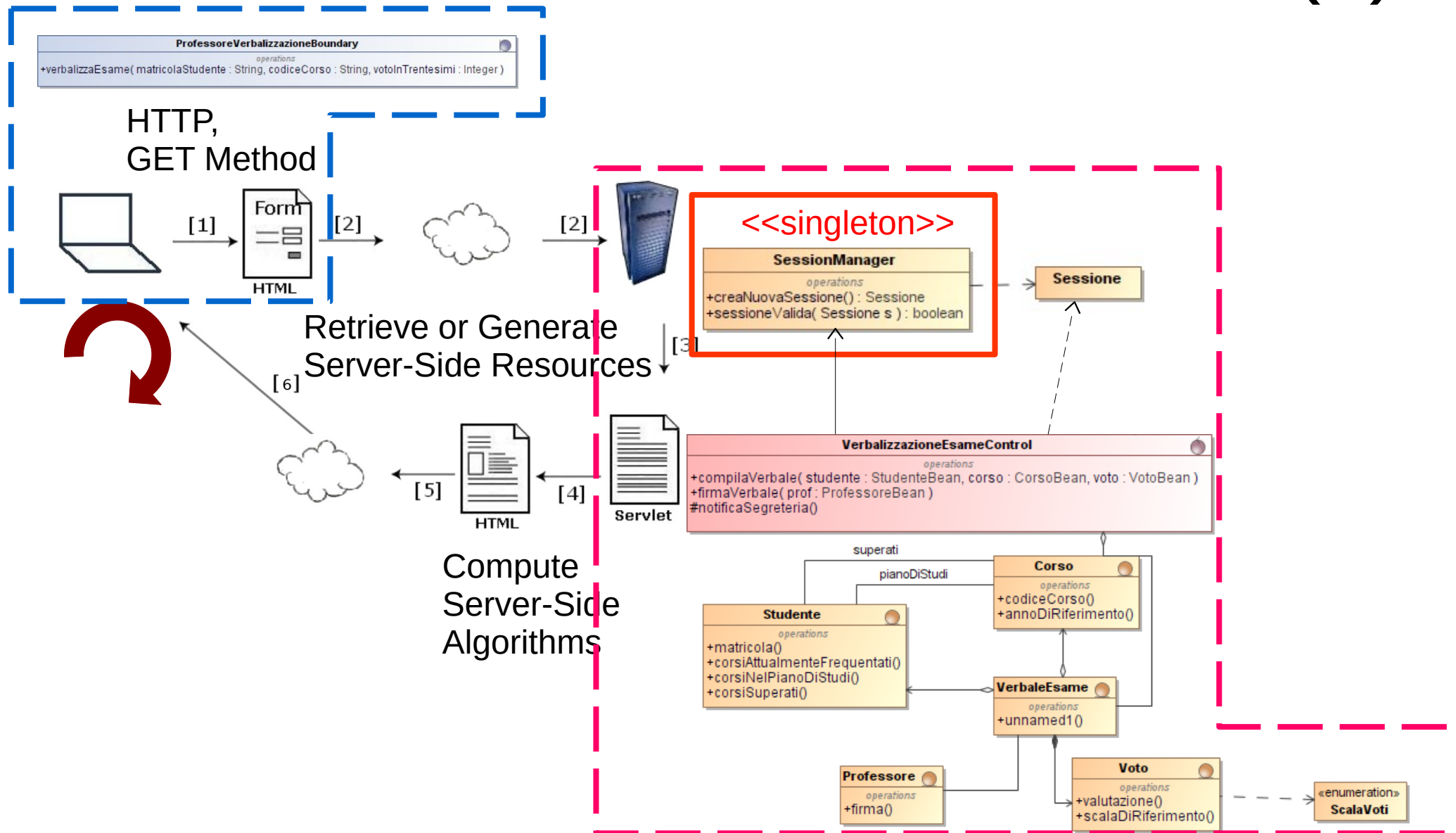
Errore:

- simili considerazioni ai casi precedenti: tutte le richieste condividono l'unica istanza di Sessione
- tutte le richieste faranno riferimento all'unica sessione esistente
 - creata in modo esclusivo o per gestire la verbalizzazione di Guybrush Threepwood o quella Elaine Marley

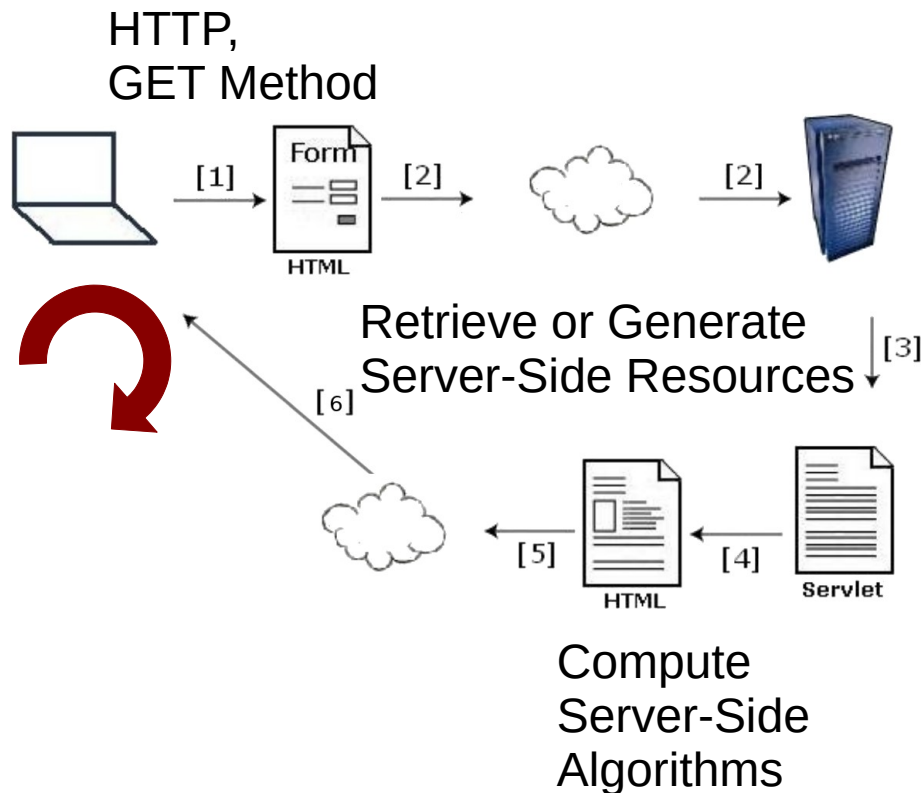
due richieste di verbalizzazione di
dell'esame di ISPW per due diversi
studenti: Guybrush Threepwood e
Elaine Marley



architetture distribuite – attenzione!!! (3)



architetture distribuite – in conclusione



- l'applicazione progettata stand-alone dovrebbe essere portata in ambito distribuito cambiando solo le parti di view
- probabilmente la versione del progetto che presenterete durante l'esame coprirà scenari single-thread e single-user
 - infatti: la trattazione esplicita dei thread non è inclusa nel corso
 - tuttavia: la progettazione della vostra soluzione dovrebbe scalare facilmente adattandosi all'ambito distribuito (almeno nella parte di realizzazione di logica applicativa)

credits

parte dei contenuti di queste slide sono stati elaborati a partire dalla presentazione del Prof. Giovanni Cantone :

“Boundary-Control-Entity Structuring Concepts for Stand-alone (Lap-top) Applications, and Web Applications”